



DOCUMENTACIÓN FINAL DEL PROYECTO

DASC Server Manager

Implementación, despliegue, test y valoración final

Integrantes

Aiman Fadli, Daniel Gálvez, Santino Colmena y
Carlos Cano

Tutor/a

Jesús Vázquez / Susana Barbero

Curso

CFGS ASIX - Proyecto final

Centro

Institut Manuel Vázquez Montalbán

Fecha de entrega

Mayo de 2026

INS MANUEL
VÁZQUEZ
MONTALBÁN



2. Índice de contenidos

2. Índice de contenidos	2
3. Definición del proyecto indicando análisis y requisitos	7
3.1 Contexto y justificación	7
3.2 Problema detectado	7
3.3 Solución propuesta	8
3.4 Objetivo general	8
3.5 Objetivos específicos	8
3.6 Requisitos funcionales	9
3.7 Requisitos no funcionales	10
3.8 Alcance actual y pendientes	10
4. Diseño	11
4.1 Diseño general de la solución	11
4.2 Diagrama lógico de red	11
4.3 Diagrama físico de red	12
4.4 Esquema de servicios y puertos	13
4.5 Arquitectura interna del panel	14
4.6 Modelo de datos de logs y alertas	15
4.7 Diseño del módulo de backups	17
4.8 Diseño de seguridad	19
4.9 Diseño Docker	21
4.10 Segmentación lógica y seguridad de red	22
4.11 Modelo de datos del historial de backups	23
4.12 Diseño de interfaz y experiencia de usuario	23
5. Implementación	24
5.1 Tecnologías utilizadas	24
5.2 Estructura del repositorio	24
5.3 Backend FastAPI	25
5.4 Login, sesión y permisos	26
5.5 Administración de usuarios y logs de autenticación	28
5.6 Módulo de terminal remota	28
5.7 Implementación de backups	28
5.8 Restauración, descarga, eliminación y automatización	29
5.9 Implementación de servicios	30
5.10 Implementación de logs y monitorización	30
5.11 Implementación de alertas Telegram	31

5.12 Instaladores y despliegue	31
5.13 Implementación Docker	32
5.14 Manual resumido de instalación por máquinas	32
5.14.1 Instalación de la máquina de base de datos	32
5.14.2 Instalación de la máquina de backups y servicios	33
5.14.3 Instalación de la máquina API	33
5.15 Implementación de Cacti y monitorización	33
6. Test	34
6.1 Estrategia de pruebas	34
6.2 Tabla de pruebas principales	34
6.3 Comandos de validación recomendados	35
6.4 Incidencias detectadas durante las pruebas	36
6.5 Pruebas específicas de backups	36
6.6 Pruebas específicas de alertas	36
7. Resultado	38
7.1 Resultado final del producto	38
7.2 Guía de uso para el usuario	38
7.3 Control de calidad del producto	39
7.4 Manual de usuario detallado por módulos	39
7.4.1 Uso del módulo de copias	39
7.4.2 Uso del módulo de restauración	39
7.4.3 Uso del módulo de servicios	39
7.4.4 Uso del módulo de logs	40
7.4.5 Uso del módulo de alertas	40
7.4.6 Uso del módulo de terminal	40
7.5 Guion breve de demostración final	40
8. Pla de sostenibilitat	41
8.1 Sostenibilidad ambiental	41
8.2 Sostenibilidad económica	41
8.3 Sostenibilidad social y legal	41
8.4 Relación con ODS	42
9. Valoració funcional	43
9.1 Objetivos alcanzados	43
9.2 Limitaciones actuales	43
9.3 Mejoras futuras	43
10. Valoració tècnica	45

10.1 Herramientas de desarrollo	45
10.2 Valoración de tecnologías	45
10.3 Alternativas técnicas	45
10.4 Seguridad técnica	46
10.5 Incidencias reales y decisiones tomadas	46
11. Valoració de la gestió	48
11.1 Metodología de trabajo	48
11.2 Planificación y reajustes	48
11.3 Herramientas de colaboración	48
11.4 Reparto de tareas	49
11.5 Documentación técnica de administración	49
11.6 Retrospectiva final del equipo	49
11.7 Criterios de cierre de documentación	50
11.8 Checklist de entrega final	50
12. Bibliografia i Webgrafia	51
Annexos	53
Annex A. Lista de capturas recomendadas	53
Annex B. Diagramas obligatorios y recomendados	53
Annex C. Plan de cierre funcional antes de entregar	55
Annex D. Capturas del proyecto	56
Captura D.1. 02 conectividad api db backups	56
Captura D.2. 04 install db ok	57
Captura D.3. 05 mariadb status puerto	57
Captura D.4. 06 binlogs show master status	57
Captura D.5. 07 usuario backup db	58
Captura D.6. 08 install backup services ok	58
Captura D.7. 11 install api ok	58
Captura D.8. 12 dasc api status	58
Captura D.9. 13 dependencias python	59
Captura D.10. 14 login	59
Captura D.11. 15 login error	60
Captura D.12. 16	60
Captura D.13. 17 admin usuarios	61
Captura D.14. 18 crear usuario limitado	61
Captura D.15. 19 usuario limitado menu	62
Captura D.16. 21 admin login logs	62

Captura D.17. 23	63
Captura D.18. 23 backup full ok	63
Captura D.19. 24 backup incremental ok	64
Captura D.20. 24 captura24	64
Captura D.21. 25 captura25 incremental	65
Captura D.22. 26 archivos backups remoto	65
Captura D.23. 26 diferencial	66
Captura D.24. 27 captura27	66
Captura D.25. 27 contenido backup	67
Captura D.26. 28 captura	67
Captura D.27. 28 descarga backup panel	68
Captura D.28. 29 eliminacion backup confirmacion	69
Captura D.29. 29 eliminacion backup confirmacion2	69
Captura D.30. 30	70
Captura D.31. 30 restore	71
Captura D.32. 30 restore preview confirm	72
Captura D.33. 31 automatizacion backups	72
Captura D.34. 33	73
Captura D.35. 36	73
Captura D.36. 36 captura36	74
Captura D.37. 37	74
Captura D.38. 38	74
Captura D.39. 38 captura38	75
Captura D.40. 39	75
Captura D.41. 39 alertas telegram(1)	76
Captura D.42. 39 alertas telegram	76
Captura D.43. 40 alertas detectar destinatario	77
Captura D.44. 41 alertadeprueba	77
Captura D.45. 41 alertas	78
Captura D.46. 41 telegram alerta prueba1	79
Captura D.47. 43	79
Captura D.48. 44	80
Captura D.49. 45 github estructura	80
Captura D.50. 46	81
Captura D.51. 47	81
Captura D.52. 48	82

Captura D.53. 49	82
Captura D.54. 50.1	82
Captura D.55. 50	82
Annex E. Diagramas adicionales	83
Diagrama E.1. diagrama 2 servicios puertos dacsc	83

3. Definición del proyecto indicando análisis y requisitos

3.1 Contexto y justificación

DASC Server Manager es un panel web de administración orientado a centralizar tareas habituales de sistemas: ejecución de copias de seguridad, gestión de servicios, consulta de logs, monitorización y alertas. El proyecto parte de una necesidad realista en pequeñas y medianas empresas: disponer de una herramienta local que permita operar sobre servidores sin depender totalmente de plataformas externas.

La propuesta se ha construido desde una visión de ciberseguridad y administración de sistemas. El objetivo no es solamente que el panel sea visualmente atractivo, sino que las acciones que se ejecuten desde la interfaz tengan una traducción real sobre la infraestructura: comandos remotos por SSH, scripts Bash, bases de datos MariaDB, servicios systemd, históricos de backups y auditoría de eventos.

Durante el desarrollo el proyecto ha evolucionado desde un MVP más limitado hacia una versión casi final. Actualmente el sistema ya integra autenticación, usuarios y permisos, logs de acceso, alertas por Telegram, terminal remota controlada, backups completos, incrementales y diferenciales basados en binlogs, restauración, descarga, eliminación y automatización básica de copias.

Evidencia en anexos: Captura recomendada: portada o dashboard general del proyecto funcionando. Ver Annex D de capturas.
--

3.2 Problema detectado

Muchas organizaciones pequeñas gestionan sus bases de datos y servicios de forma manual. Esto provoca dependencia de comandos de terminal, errores humanos, poca trazabilidad y dificultad para saber quién ha ejecutado una acción o cuándo se ha producido un fallo.

Otro problema importante es la dependencia de terceros. Las soluciones cloud o los servicios gestionados externos pueden ser útiles, pero no siempre encajan con empresas que quieren

mantener sus datos en local, reducir costes recurrentes o tener control directo de su infraestructura.

3.3 Solución propuesta

La solución propuesta es un panel web desplegado sobre un servidor API que actúa como punto central de control. Desde este panel el usuario puede acceder a diferentes módulos según sus permisos: backups, logs, servicios, alertas, terminal y administración. La API no ejecuta todas las acciones localmente, sino que se comunica con servidores específicos mediante SSH y con la base de datos mediante MariaDB.

- Servidor API / panel web: ejecuta FastAPI, sirve plantillas HTML y controla la sesión de los usuarios.
- Servidor de base de datos: aloja MariaDB, la base employees de ejemplo, la base dasc_logs y los binlogs necesarios para copias incrementales y diferenciales.
- Servidor de backups y servicios: almacena las copias, ejecuta backups_api.sh, restore_api.sh y servicios_api.sh, y permite controlar servicios mediante systemctl.
- Sistema de alertas: utiliza SQLite para reglas y destinatarios, y Telegram como canal principal de notificación.
- Versión Docker: reproduce la arquitectura del laboratorio mediante contenedores con direcciones fijas dentro de una red bridge.

3.4 Objetivo general

Desarrollar e implantar una herramienta local de administración de servidores que permita gestionar copias de seguridad, servicios, logs, monitorización y alertas desde un panel web sencillo, manteniendo una separación clara entre interfaz, base de datos, almacenamiento de copias y ejecución de tareas administrativas.

3.5 Objetivos específicos

- Crear un panel web accesible desde navegador con autenticación de usuarios.
- Implementar roles y permisos para limitar el acceso a módulos sensibles.
- Ejecutar backups completos, incrementales y diferenciales desde el panel.

- Registrar un historial de backups con ID, fecha, tipo, base de datos, fichero y referencia a su copia base.
- Permitir descarga, restauración y eliminación de backups con confirmaciones previas.
- Gestionar servicios remotos mediante acciones start, stop y restart.
- Registrar eventos del panel en una base de datos de logs.
- Añadir alertas por Telegram para eventos críticos o informativos.
- Preparar instaladores para API, base de datos, backups/servicios, Cacti, actualización y desinstalación.
- Mantener una versión Docker reproducible para demostraciones y pruebas rápidas.

3.6 Requisitos funcionales

Código	Requisito	Descripción
RF01	Login y sesión	El usuario debe autenticarse antes de acceder al panel.
RF02	Usuarios y permisos	El administrador puede crear usuarios y asignar módulos permitidos.
RF03	Backups	El sistema permite lanzar copias full, incremental y differential.
RF04	Historial de copias	El panel muestra las últimas copias con estado y metadatos.
RF05	Restauración	El usuario con permiso puede restaurar una copia mediante una cadena calculada desde el full base.
RF06	Descarga	El usuario puede descargar un backup validado desde el directorio permitido.
RF07	Eliminación	El sistema calcula dependencias y elimina en cascada cuando sea necesario.
RF08	Servicios	El usuario puede listar, iniciar, parar y reiniciar servicios remotos.
RF09	Logs	El panel registra eventos de acceso, backups, servicios, alertas, terminal y administración.
RF10	Alertas	El panel permite configurar destinatarios Telegram y enviar pruebas.
RF11	Terminal	El panel permite ejecutar comandos controlados en máquinas autorizadas.
RF12	Instalación	El proyecto incluye scripts para instalar y actualizar los roles principales.

3.7 Requisitos no funcionales

Código	Tipo	Descripción
RNF01	Usabilidad	La interfaz debe ser clara y permitir ejecutar operaciones en pocos clics.
RNF02	Seguridad	Las rutas deben estar protegidas por sesión y permisos.
RNF03	Trazabilidad	Las acciones relevantes deben quedar registradas.
RNF04	Separación de responsabilidades	La API, la base de datos y el servidor de backups deben tener roles diferenciados.
RNF05	Mantenibilidad	Los scripts deben estar separados por rol y documentados.
RNF06	Reproducibilidad	Debe existir una instalación automatizada y una versión Docker de apoyo.

3.8 Alcance actual y pendientes

El proyecto se encuentra en una fase muy avanzada. La mayoría de funcionalidades previstas para la versión final están implementadas o preparadas para validación final con capturas. La documentación se completa con diagramas y anexos para dejar evidencias sin sobrecargar la memoria principal.

Elemento	Estado actual	Decisión para la documentación
Panel FastAPI	Implementado	Mostrar como funcionalidad principal.
Login y permisos	Implementado	Validar con usuario administrador y usuario limitado.
Backups full/incremental/differential	Implementado con binlogs en scripts actuales	Incluir pruebas con historial y base_id.
Restauración, descarga y eliminación	Implementado en rutas y scripts	Validar con capturas antes de entregar.
Alertas Telegram	Implementado	Validar con captura de destinatarios, prueba enviada y mensaje recibido.
Terminal remota	Implementada	Presentar como módulo avanzado con permisos.

4. Diseño

4.1 Diseño general de la solución

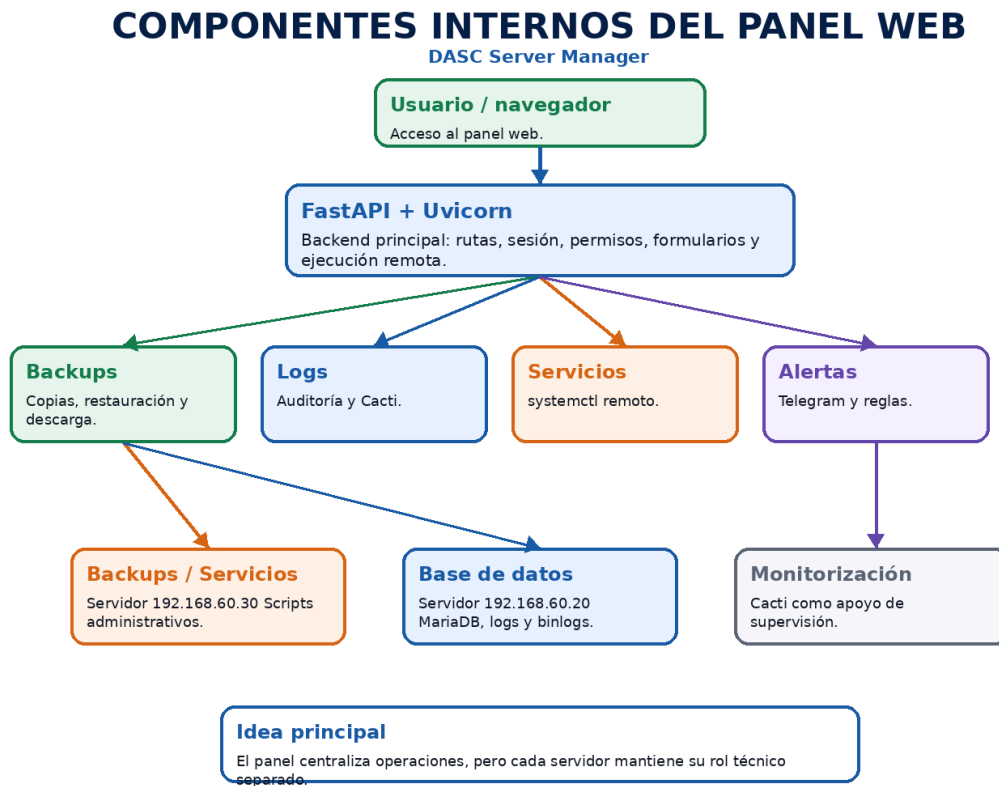


Figura 1. Componentes internos del panel web.

El diseño de DASC Server Manager se basa en una arquitectura distribuida. En vez de instalar todas las funciones en una única máquina, el proyecto separa la interfaz web, la base de datos y el servidor que ejecuta las tareas administrativas. Esta separación facilita la defensa técnica del proyecto porque reproduce una estructura más realista que un único servidor monolítico.

4.2 Diagrama lógico de red

La red de laboratorio se organiza alrededor de la subred 192.168.60.0/24. El flujo principal es usuario -> API -> servidor de backups/servicios -> base de datos. La API actúa como punto de

entrada y no se recomienda exponer directamente MariaDB ni el servidor de backups al usuario final.

Equipo	Rol	Dirección IP	Servicios principales
API / Panel	Interfaz web y backend	192.168.60.10	FastAPI/Uvicorn en 8000, cliente SSH, acceso a logs
Base de datos	MariaDB y logs	192.168.60.20	MariaDB 3306, binlogs, dasc_logs, employees
Backups + Servicios	Ejecución remota	192.168.60.30	SSH 22, backups_api.sh, restore_api.sh, servicios_api.sh, cron
Cliente / administrador	Navegador web	DHCP o IP manual	HTTP hacia API

4.3 Diagrama físico de red

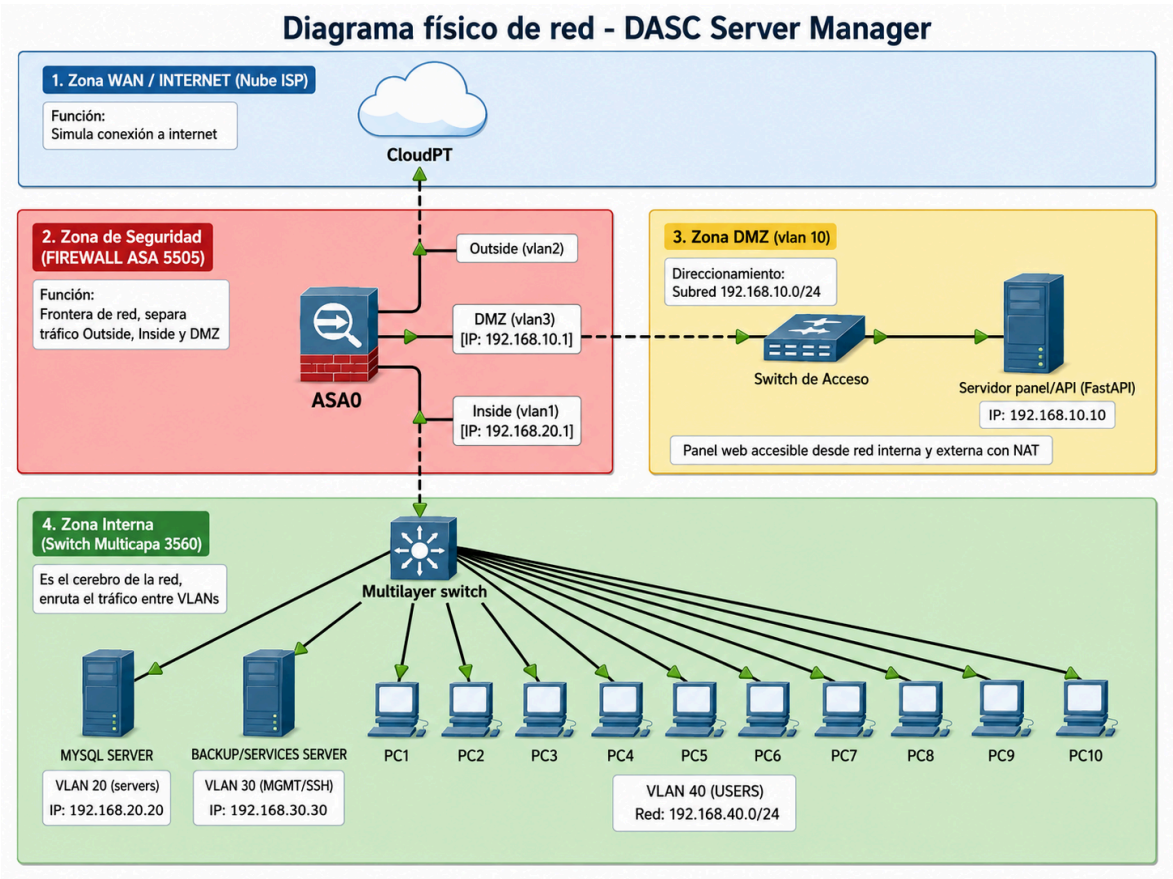


Figura 2. Diagrama físico de red.

El diagrama físico debe mostrar las máquinas virtuales o servidores reales conectados al switch o red interna del laboratorio. En la memoria final se recomienda representar también el equipo

del administrador, la salida NAT si se utiliza para instalar paquetes y la red interna usada por las VMs para comunicarse entre sí.

Evidencia en anexos: Diagrama físico de red con las tres máquinas y el equipo administrador. Ver Annex D de capturas.

4.4 Esquema de servicios y puertos

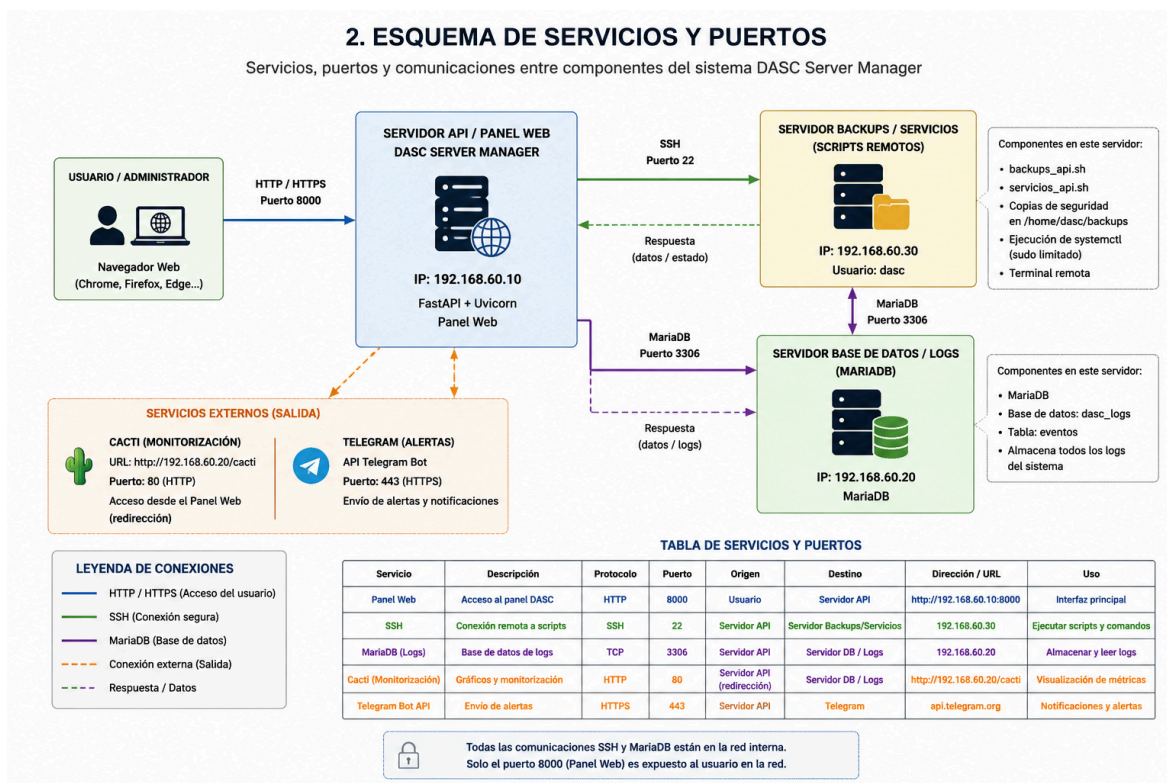


Figura 3. Esquema de servicios y puertos.

Servicio	Puerto	Origen	Destino	Uso
HTTP FastAPI	8000	Cliente	API	Acceso al panel web.
SSH	22	API	Backups/Servicios	Ejecución de scripts administrativos.
SSH	22	API	Base de datos	Terminal remota controlada y comprobaciones.
MariaDB	3306	Backups/Servicios	Base de datos	mysqldump, mysqlbinlog y restauración.
Cacti	80/8081 según despliegue	Cliente	Cacti	Monitorización web.
Telegram HTTPS	443	API	API de Telegram	Envío de alertas.

4.5 Arquitectura interna del panel

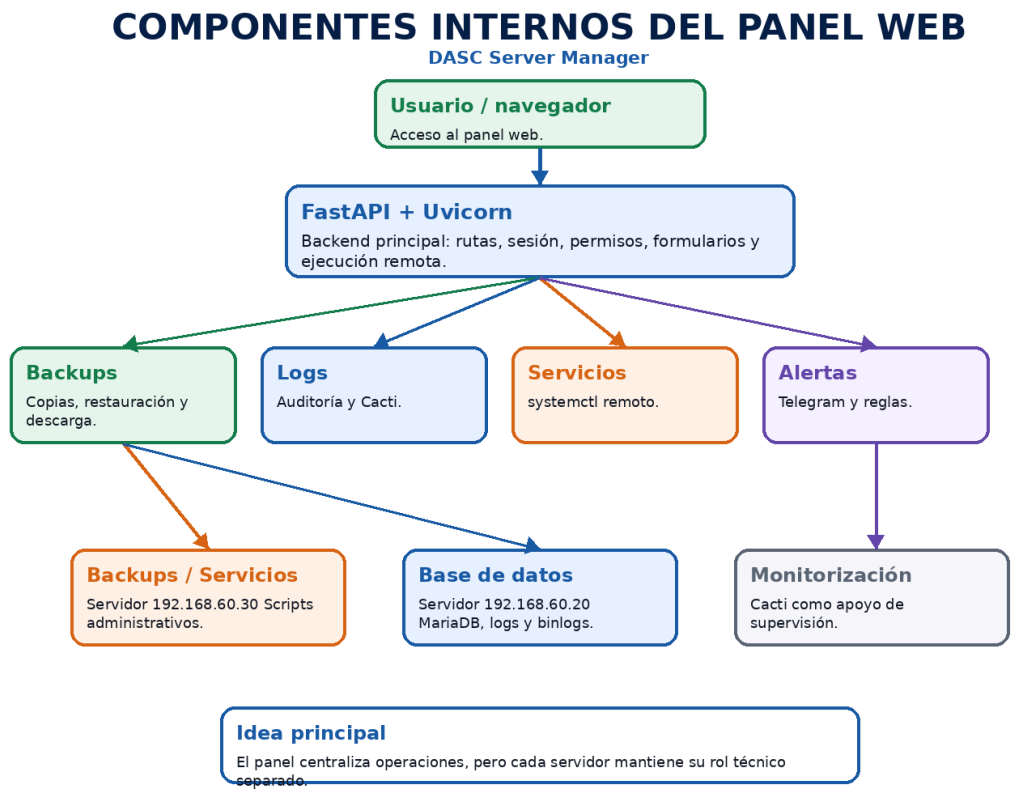


Figura 4. Arquitectura interna del panel web.

Internamente, el panel sigue una organización típica de aplicación FastAPI con plantillas Jinja2. main.py centraliza las rutas y la lógica de control, templates contiene las páginas HTML y static contiene los estilos, imágenes y documentación accesible desde la web.

Elemento	Responsabilidad
main.py	Rutas HTTP, sesión, permisos, logs, backups, servicios, terminal y alertas.
templates/login.html	Pantalla de acceso al panel.
templates/index.html	Dashboard principal y accesos según permisos.
templates/backups.html	Formulario e historial de copias.
templates/logs.html	Tabla de eventos y acceso a Cacti.
templates/servicios.html	Listado y control de servicios remotos.
templates/alertas.html	Configuración de Telegram, reglas y destinatarios.
templates/terminal.html	Terminal remota con máquinas autorizadas.
static/css	Estilos generales y específicos del panel.

Elemento	Responsabilidad
data/users.json	Usuarios creados desde el panel y permisos asignados.
data/alerts.db	Base SQLite para alertas, reglas, entregas y destinatarios.

4.6 Modelo de datos de logs y alertas

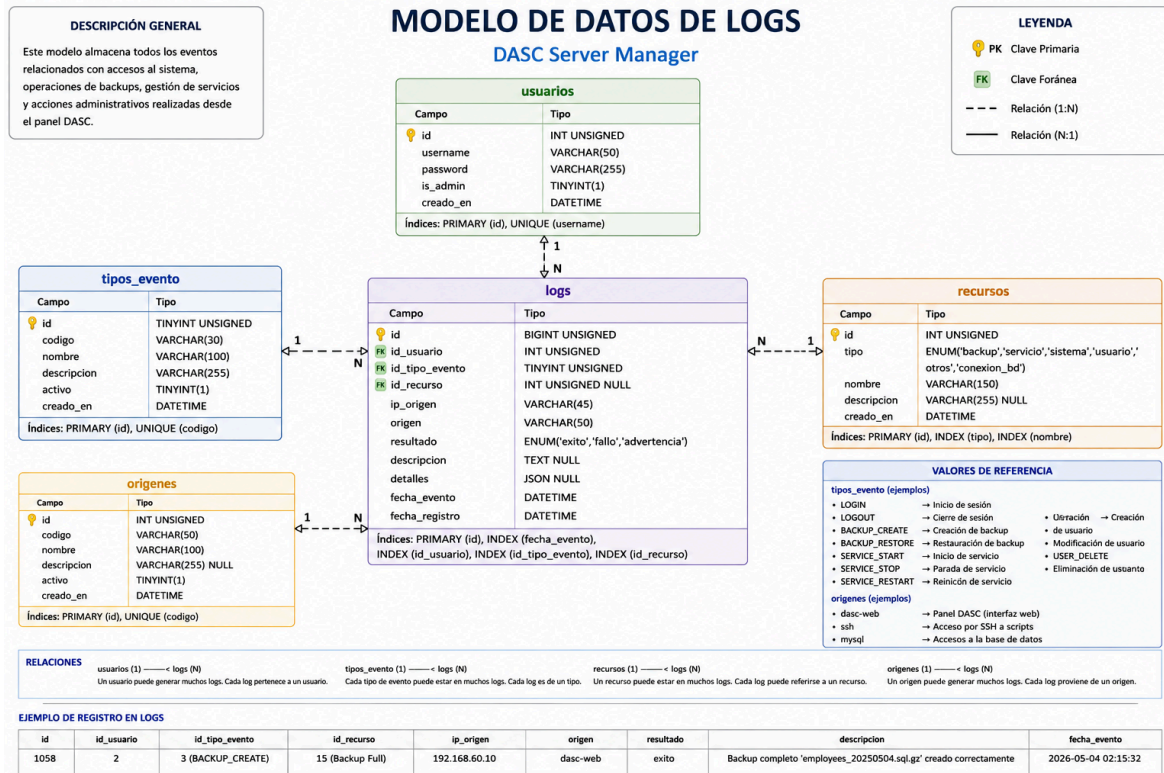


Figura 5. Modelo de datos de logs.

El sistema de logs combina varias fuentes: eventos del panel en MariaDB, registros de login en JSON local y eventos/entregas de alertas en SQLite. Esta separación evita mezclar datos operativos con datos de configuración del canal de alertas.

Flujo de logs, auditoría y alertas

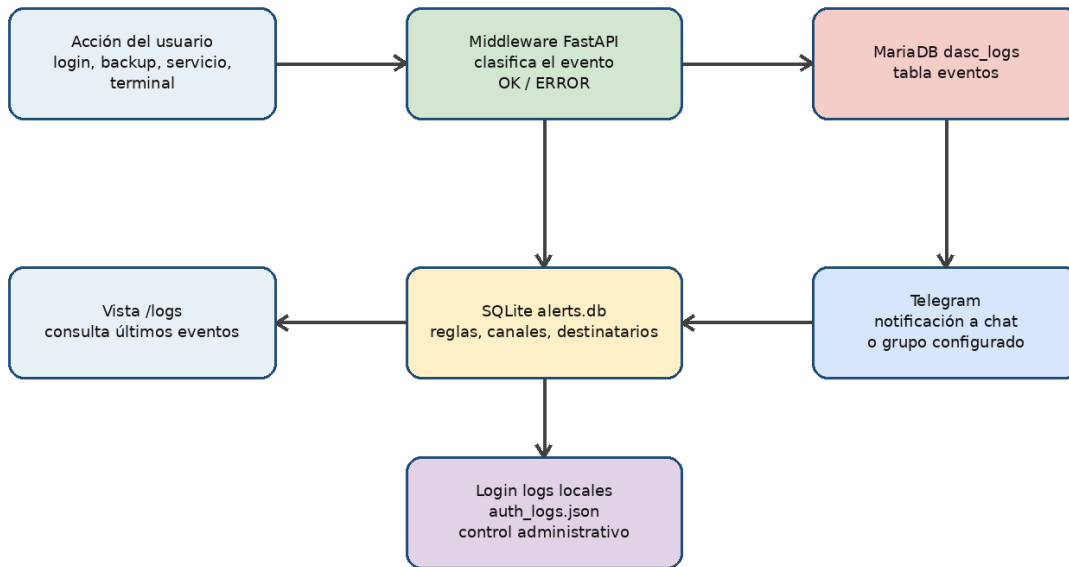


Tabla / fichero	Campos principales	Uso
eventos	id, fecha, origen, tipo, usuario, ip_origen, recurso, resultado, detalle	Auditoría general del panel.
auth_logs.json	fecha, acción, usuario, resultado, IP, user-agent, rol, detalle	Revisión administrativa de accesos.
alert_channels	code, name, is_enabled, token, destination	Canales de alertas disponibles.
alert_rules	event_code, severity, channel_code, is_enabled	Reglas que activan alertas.
alert_recipients	name, destination, recipient_type, is_active, is_default	Chats o grupos de Telegram.
alert_deliveries	event_id, channel_code, recipient_id, status, response_text	Resultado de cada envío.

4.7 Diseño del módulo de backups

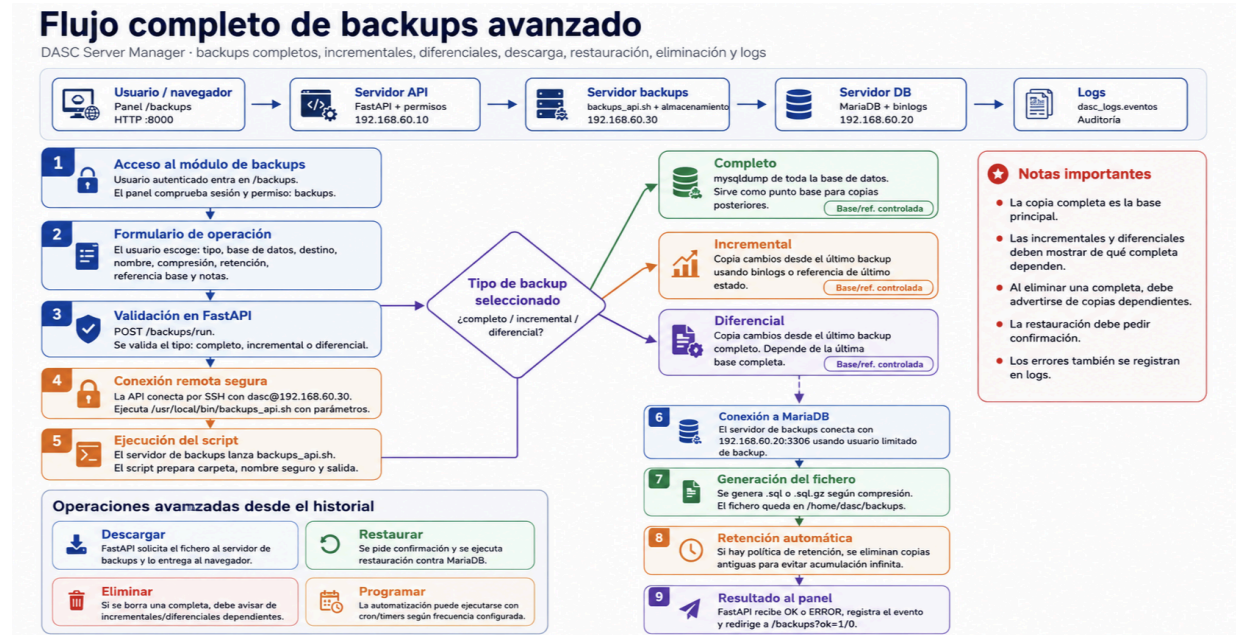
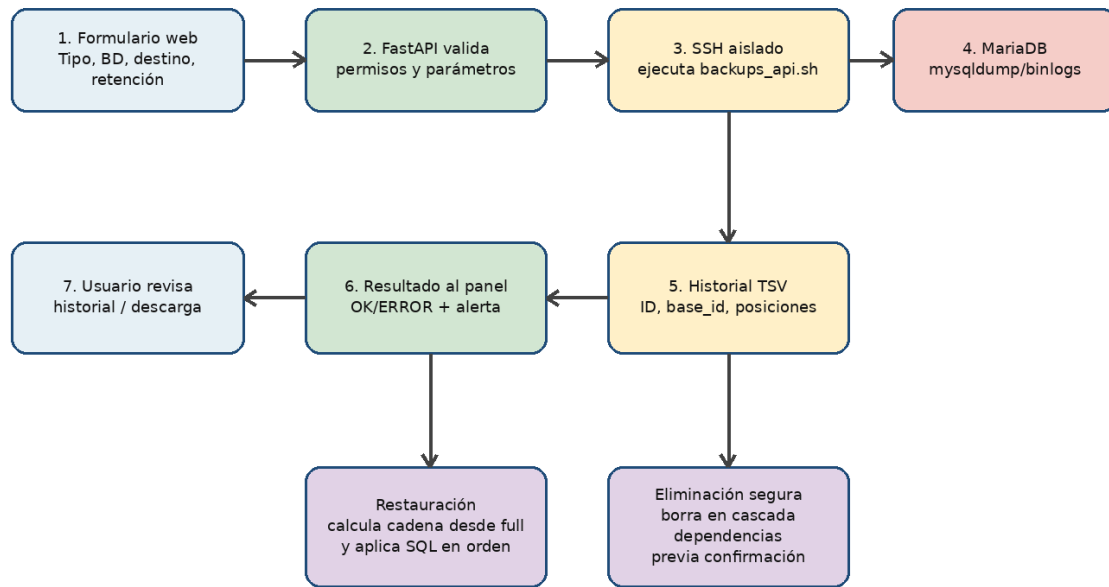


Figura 6. Flujo completo de backups avanzado.

El diseño del módulo de backups se apoya en un historial TSV mantenido en el servidor de backups. Cada copia recibe un ID y, si no es completa, guarda el ID de la copia base. Esto permite saber de qué full depende una copia diferencial y de qué copia anterior depende una incremental.

Flujo de copias de seguridad y restauración



Tipo de copia	Funcionamiento diseñado	Dependencia
Full	Exporta la base completa mediante mysqldump/mariadb-dump e incluye coordenadas de binlog.	No depende de otra copia.
Incremental	Extrae binlogs desde la última copia válida de la misma base de datos.	Depende del último backup, sea full o incremental.
Diferencial	Extrae binlogs desde el último backup completo.	Depende del último full.
Restauración	Calcula la cadena desde el full hasta el ID seleccionado y aplica los SQL en orden.	Depende de la integridad del historial.
Eliminación	Calcula dependencias y advierte si hay copias afectadas.	Puede borrar en cascada.

4.8 Diseño de seguridad

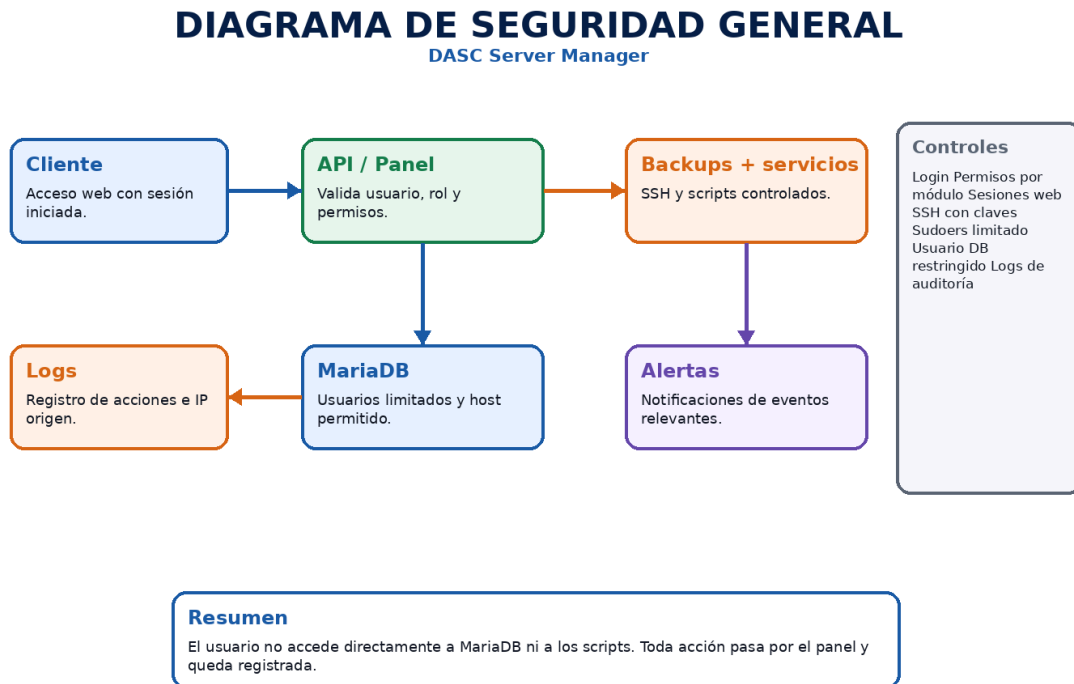
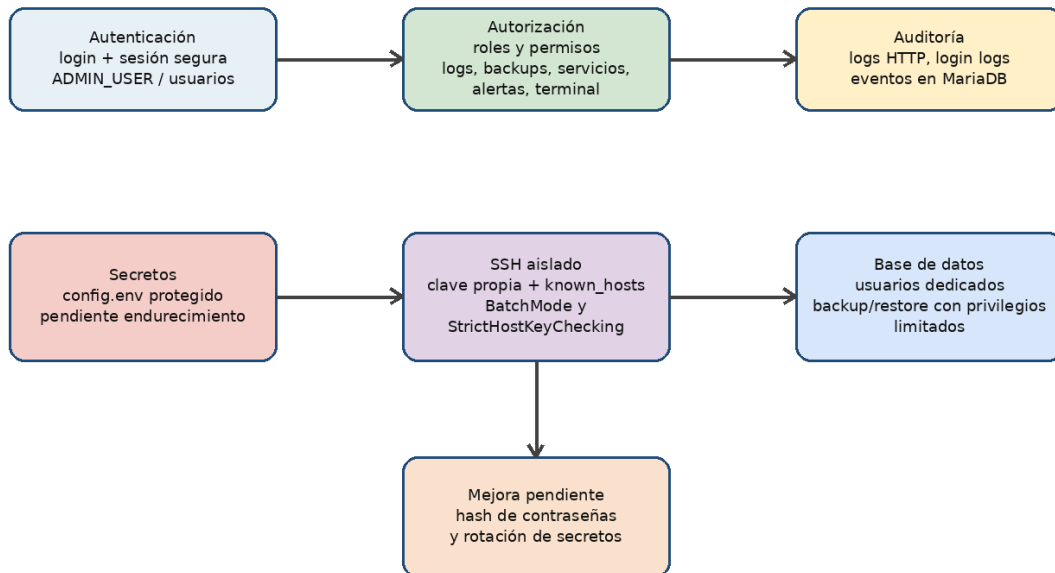


Figura 7. Diagrama de seguridad general.

La seguridad se diseña por capas. El usuario primero debe autenticarse. Después, el panel muestra y permite ejecutar únicamente los módulos autorizados. Para ejecutar acciones remotas se utiliza una clave SSH propia del proyecto dentro de `/opt/dasc/api/.ssh`, con `known_hosts` separado. Las operaciones contra MariaDB se realizan con usuarios específicos de backup y restauración.

Diseño de seguridad y controles principales



4.9 Diseño Docker

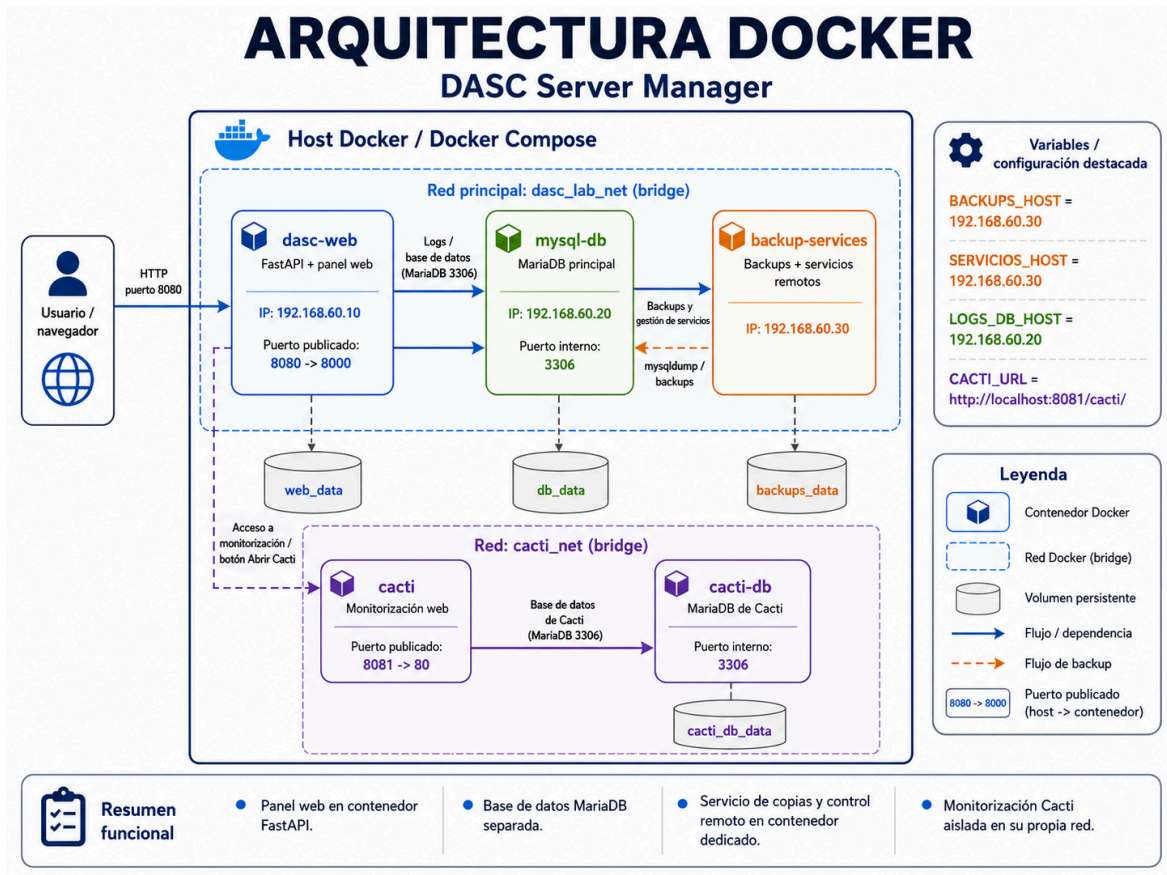
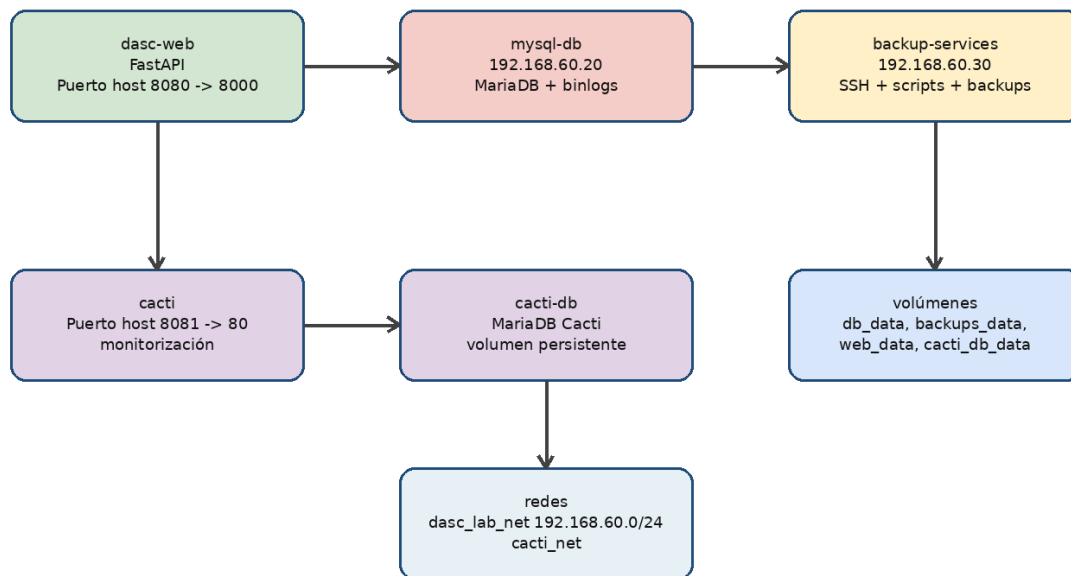


Figura 8. Arquitectura Docker.

Además del despliegue en tres máquinas, se ha preparado una versión Docker fiel al proyecto. Esta versión reproduce las IP principales dentro de una red bridge y permite demostrar el producto sin depender de una OVA concreta. No sustituye la explicación de ASIX por máquinas, pero ayuda a validar instalación, estructura y dependencias.

Versión Docker fiel al entorno de máquinas virtuales



4.10 Segmentación lógica y seguridad de red

Aunque el laboratorio principal se ha simplificado con la red 192.168.60.0/24 para facilitar las pruebas, el diseño teórico de ASIX debe explicar cómo se separaría el sistema en una empresa real. La segmentación permite que los usuarios no accedan directamente a la base de datos ni al servidor de backups, sino únicamente al panel expuesto.

Zona	Subred propuesta	Elementos	Política recomendada
DMZ / Panel	192.168.10.0/24	Servidor API, reverse proxy si se añade	Acceso desde usuarios solo a HTTP/HTTPS del panel.
Datos	192.168.20.0/24	MariaDB, base employees, dasc_logs	Solo conexiones desde servidor de backups/API según necesidad.
Gestión	192.168.30.0/24	Servidor de backups, SSH, scripts administrativos	Acceso limitado desde API y administradores.
Usuarios	192.168.40.0/24	PCs de administradores y empleados	Sin acceso directo a MariaDB ni scripts.
Monitorización	192.168.50.0/24	Cacti, logs y herramientas auxiliares	Acceso de lectura/controlado.

En la memoria principal se puede presentar esta segmentación como diseño objetivo, aunque la demo final use la red de laboratorio 192.168.60.0/24. Esta distinción es importante: el prototipo valida la funcionalidad y el diseño explica cómo se endurecería en una implantación real.

4.11 Modelo de datos del historial de backups

El historial de backups es una pieza central porque permite transformar ficheros sueltos en una cadena restaurable. Sin historial, el panel podría listar archivos, pero no sabría qué incremental depende de qué full ni qué archivos habría que restaurar para volver a un punto concreto.

Campo	Descripción	Ejemplo
id	Identificador único incremental de la copia.	12
created_at	Fecha y hora de creación.	2026-05-15 10:30:00
type	Tipo de backup.	full / incremental / differential
db	Base de datos afectada.	employees
file	Ruta absoluta del fichero generado.	/home/dasc/backups/backup.sql.gz
base_id	ID de la copia base si existe.	10
start_file/start_pos	Inicio del rango de binlog.	dasc-bin.000003 / 245
end_file/end_pos	Fin del rango de binlog.	dasc-bin.000004 / 891
notes	Notas introducidas desde el panel.	backup antes de actualización

Este diseño hace posible que la interfaz muestre de forma comprensible si una copia es independiente o si depende de otras. También permite que la eliminación advierta cuando borrar una copia completa afectaría a incrementales o diferenciales posteriores.

4.12 Diseño de interfaz y experiencia de usuario

La interfaz se ha diseñado con una estructura común: cabecera superior, menú lateral y área principal de contenido. Esta repetición facilita que el usuario entienda dónde se encuentra y qué acciones puede realizar. El menú lateral se genera dinámicamente según permisos, evitando mostrar módulos que el usuario no puede utilizar.

- Cabecera con identidad visual del proyecto y centro educativo.
- Menú lateral con Panel, Copias, Logs, Servicios, Alertas, Terminal y Administración según permisos.
- Tarjetas en el dashboard para resumir funcionalidades disponibles.
- Tablas para históricos de backups, logs y servicios.
- Alertas visuales OK/ERROR tras acciones administrativas.
- Formularios simples para ejecutar acciones sin obligar al usuario a escribir comandos largos.

Evidencia en anexos: Captura comparativa del dashboard con administrador y usuario limitado para demostrar menú dinámico. Ver Annex D de capturas.

5. Implementación

5.1 Tecnologías utilizadas

Tecnología	Uso en el proyecto	Valoración
Ubuntu Server	Sistema base de las máquinas virtuales.	Estable, conocido y adecuado para ASIX.
Python	Lenguaje principal del backend.	Permite integrar web, SSH, JSON, SQLite y MariaDB.
FastAPI	Framework web del panel.	Ligero, rápido y cómodo para rutas y formularios.
Uvicorn	Servidor ASGI.	Adecuado para servir FastAPI como servicio systemd.
Jinja2	Plantillas HTML dinámicas.	Permite generar menús según permisos.
HTML/CSS	Interfaz del panel.	Suficiente para el MVP y fácilmente modificable.
MariaDB/MySQL	Base employees, logs, usuarios DB de backup y restore.	Adecuado para backups con mysqldump y binlogs.
Bash	Scripts de instalación, backups, restauración y servicios.	Directo para administración de sistemas.
SSH	Comunicación entre API y máquinas remotas.	Realista para administración segura.
SQLite	Base local de alertas.	Sencilla para reglas y destinatarios sin añadir otro servidor.
Telegram Bot API	Canal de alertas.	Fácil de demostrar y útil para eventos críticos.
Docker Compose	Entorno reproducible de demostración.	Facilita levantar el stack completo.
Cacti	Monitorización.	Complementa logs con una herramienta de supervisión.

5.2 Estructura del repositorio

El repositorio se organiza principalmente en deploy/, separado por roles. Esta estructura ayuda a ejecutar cada instalador en la máquina correspondiente y evita mezclar scripts de API, base de datos y servidor de backups.

```
deploy/  
  api/  
    install_dasc_api.sh  
    uninstall_dasc_api.sh  
    update_dasc_api.sh  
  cacti/  
  package/  
    main.py  
    config.env  
    requirements.txt  
    templates/  
    static/
```

```

    data/
backup-services/
  install_backup_services.sh
  uninstall_backup_services.sh
package/
  backups_api.sh
  restore_api.sh
  servicios_api.sh
db/
  install_db.sh
  uninstall_db.sh
docker/
  web/
  db/
  backup-services/
  cacti/
  cacti-db/

```

5.3 Backend FastAPI

El backend principal se encuentra en main.py. Este archivo carga variables desde config.env, monta la carpeta static, configura plantillas Jinja2 y añade middleware de sesión y logging.

Las rutas se dividen por módulos: login/logout, panel principal, administración de usuarios, login logs, terminal, logs, alertas, servicios y backups. Esta organización permite que el panel crezca sin cambiar completamente la estructura inicial.

El middleware AuthAndLogMiddleware tiene dos responsabilidades: bloquear accesos no autenticados y registrar las peticiones relevantes. De esta manera, la auditoría no depende de que cada ruta recuerde registrar manualmente la acción.

Ruta	Método	Función
/login	GET/POST	Mostrar formulario y validar credenciales.
/	GET	Dashboard principal con tarjetas según permisos.
/admin/usuarios	GET/POST	Crear y listar usuarios con permisos.
/admin/login-logs	GET	Revisar inicios de sesión.
/terminal	GET/POST API	Ejecutar comandos autorizados.
/logs	GET	Mostrar eventos del panel.
/alertas	GET/POST	Configurar Telegram, reglas y destinatarios.
/servicios	GET/POST	Listar y controlar servicios remotos.
/backups	GET/POST/API	Ejecutar, automatizar, descargar, restaurar y eliminar backups.

5.4 Login, sesión y permisos

MODELO DE USUARIOS Y PERMISOS

DASC Server Manager

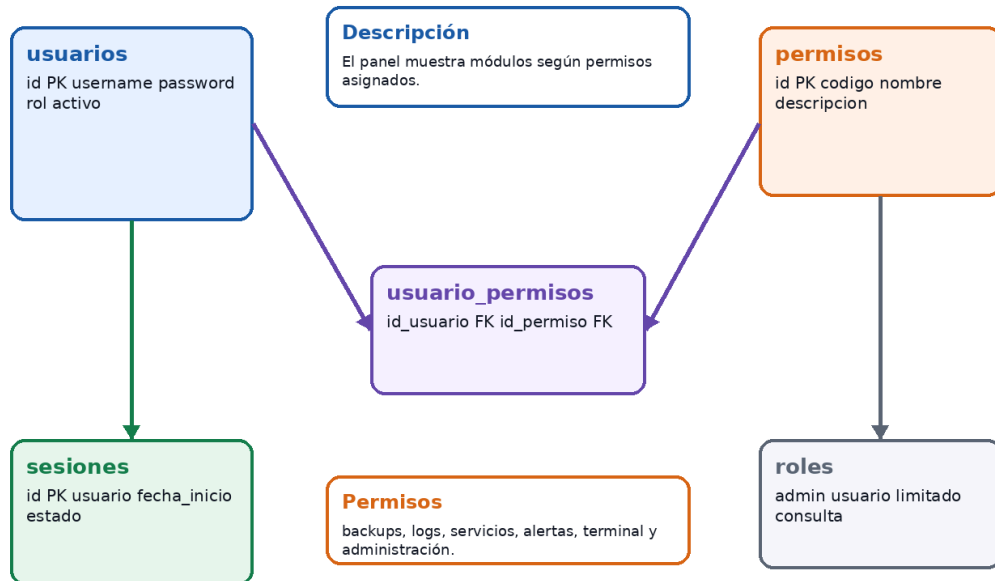


Figura 10. Modelo de usuarios y permisos.

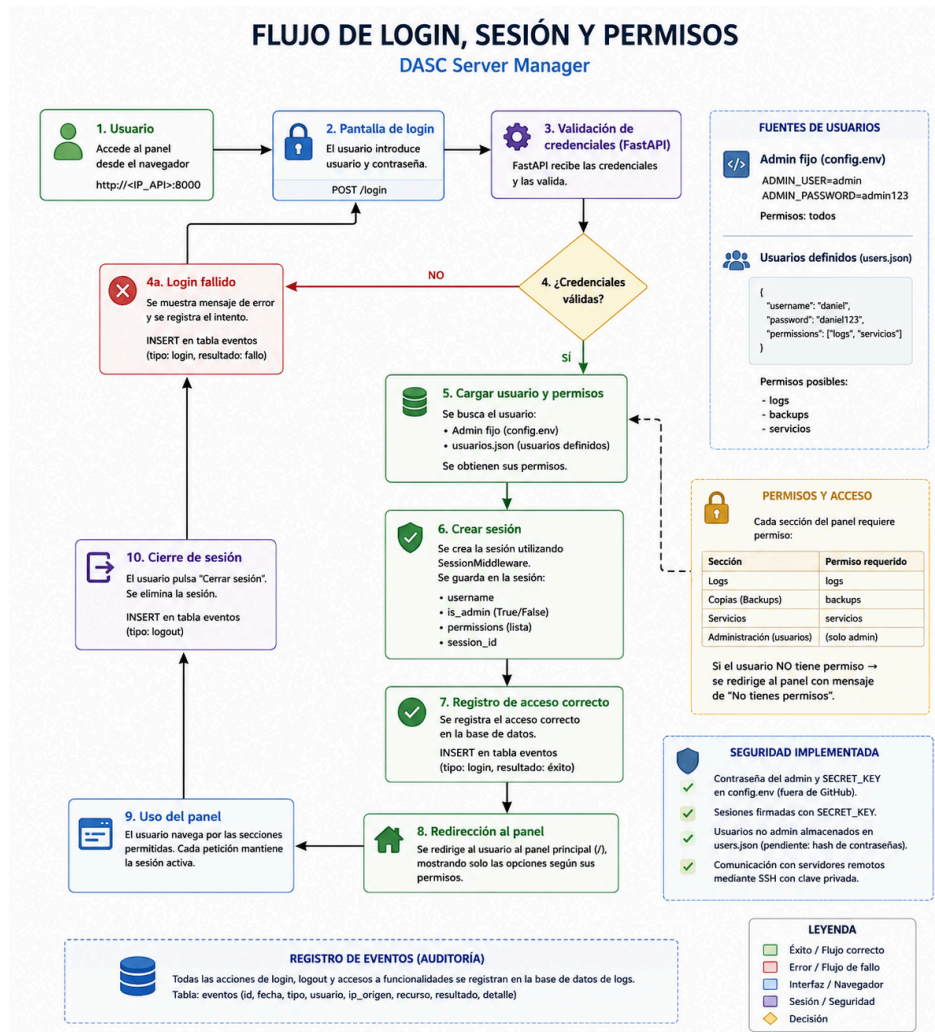


Figura 9. Flujo de login, sesión y permisos.

El login utiliza un usuario administrador definido por variables de entorno y usuarios adicionales almacenados en data/users.json. El administrador tiene todos los permisos; los usuarios normales solo pueden ver los módulos que se les asignen.

- Permiso logs: acceso a la vista de eventos y monitorización.
- Permiso backups: ejecución y gestión de copias.
- Permiso servicios: control de servicios systemd remotos.
- Permiso alertas: configuración de Telegram, reglas y destinatarios.
- Permiso terminal: acceso a la terminal remota controlada.

Evidencia en anexos: Pantalla de login con acceso correcto e incorrecto. Ver Annex D de capturas.

Evidencia en anexos: Panel con usuario administrador y panel con usuario limitado. Ver Annex D de capturas.

5.5 Administración de usuarios y logs de autenticación

La sección de administración permite crear usuarios y asignar permisos. Además, se ha añadido un registro específico de autenticación en `auth_logs.json` para revisar intentos correctos e incorrectos, cierres de sesión y datos de origen como IP y user-agent. Esta mejora aporta trazabilidad ante accesos sospechosos.

Acción	Registro generado	Utilidad
Login correcto	usuario, IP, rol, fecha	Saber quién accedió y cuándo.
Login incorrecto	usuario intentado, IP, user-agent	Detectar fuerza bruta o errores repetidos.
Logout	usuario e IP	Completar trazabilidad de sesión.
Acceso bloqueado	ruta e IP	Detectar intentos de entrar sin permiso.

5.6 Módulo de terminal remota

La terminal remota se implementa como un módulo avanzado que permite ejecutar comandos en máquinas definidas previamente. Para reducir riesgos, el módulo se protege mediante permiso específico y no debería entregarse a usuarios sin perfil técnico. Se usa como apoyo para administración y demostración, pero requiere especial cuidado en producción.

- Máquina local/API: comandos limitados al servidor del panel.
- Máquina de base de datos: conexión SSH hacia 192.168.60.20.
- Máquina de backups: conexión SSH hacia 192.168.60.30.
- Tiempo máximo de ejecución configurable mediante `TERMINAL_TIMEOUT`.
- Longitud máxima de comando configurable mediante `TERMINAL_MAX_COMMAND_LENGTH`.

Evidencia en anexos: Terminal remota mostrando hostname, ip a o systemctl status de una máquina autorizada. Ver Annex D de capturas.
--

5.7 Implementación de backups

El módulo de backups ha evolucionado de un simple `mysqldump` completo a un sistema con historial, bloqueo de ejecución, soporte para binlogs y dependencias entre copias. El script `backups_api.sh` valida parámetros, comprueba herramientas disponibles, usa `.my.cnf` para credenciales y registra cada copia en `history.tsv`.

- Bloqueo con flock para evitar dos backups simultáneos.
- Validación de tipo full, incremental o differential.
- Uso de mysqldump/mariadb-dump para full.
- Uso de mysqlbinlog/mariadb-binlog para incrementales y diferenciales.
- Registro de ID, tipo, DB, ruta, base_id, start_file, start_pos, end_file y end_pos.
- Compresión gzip opcional.
- Retención de copias antiguas si se configura.

Ejemplo de ejecución desde la API:

```
ssh -i /opt/dasc/api/.ssh/id_rsa_dasc dasc@192.168.60.30 /usr/local/bin/backups_api.sh
full employees /home/dasc/backups backup-YYYYMMDD-HHMM.sql gzip 7 "" "backup manual"
```

Evidencia en anexos: Formulario de backups con tipo full, incremental y diferencial. Ver Annex D de capturas.

Evidencia en anexos: Salida OK de backup y archivo generado en /home/dasc/backups. Ver Annex D de capturas.

5.8 Restauración, descarga, eliminación y automatización

La restauración se implementa mediante restore_api.sh. Antes de restaurar, el script calcula la cadena necesaria desde la copia full hasta la copia objetivo, valida que todos los archivos existan y crea una copia preventiva pre-restore antes de modificar la base. Esto reduce el riesgo de pérdida durante una restauración.

Funcionalidad	Implementación	Medida de seguridad
Descarga	Ruta /backups/download/{backup_id}	Valida que la ruta pertenece a /home/dasc/backups y no permite descargar el directorio completo.
Restauración	preview + confirm	Muestra cadena de restauración y exige confirmación SI.
Eliminación	preview + confirm	Calcula dependencias y elimina en cascada solo tras confirmación.
Automatización	cron remoto con marcador DASC_BACKUP_AUTO	Permite crear y borrar tareas identificables sin destruir el crontab completo.
Evidencia en anexos: Vista previa de restauración con cadena de backups. Ver Annex D de capturas.		
Evidencia en anexos: Vista previa de eliminación indicando dependencias afectadas. Ver Annex D de capturas.		

5.9 Implementación de servicios

El módulo de servicios utiliza servicios_api.sh en el servidor 192.168.60.30. La API solicita por SSH el listado de unidades systemd y presenta una tabla con el estado. Para acciones de control, el script admite start, stop y restart. El usuario dasc tiene reglas sudoers limitadas a systemctl para esas acciones.

```
/usr/local/bin/servicios_api.sh list
/usr/local/bin/servicios_api.sh restart cron.service
ssh dasc@192.168.60.30 /usr/local/bin/servicios_api.sh list | head
```

Evidencia en anexos: Tabla de servicios con estados active/inactive y botones de acción. Ver Annex D de capturas.

5.10 Implementación de logs y monitorización

Los eventos del panel se almacenan en MariaDB dentro de la base dasc_logs. La vista /logs consulta los últimos eventos y muestra información de auditoría: fecha, origen, tipo, usuario, IP, recurso, resultado y detalle. Además, desde la misma vista se puede abrir Cacti para monitorización.

Tipo de evento	Ejemplo de recurso	Resultado esperado
login	POST /login	OK o ERROR según credenciales.
acceso	GET /	OK si el usuario accede al panel.
backup	POST /backups/run	OK si la copia se genera.
servicio	POST /servicios/accion	OK si systemctl responde correctamente.
alerta	POST /alertas/test	OK si Telegram confirma envío.
terminal	POST /api/terminal/run	OK si el comando autorizado finaliza.
admin	POST /admin/usuarios	OK si se crea o elimina usuario.
Evidencia en anexos: Vista /logs con eventos después de ejecutar login, backup y servicio. Ver Annex D de capturas.		

5.11 Implementación de alertas Telegram

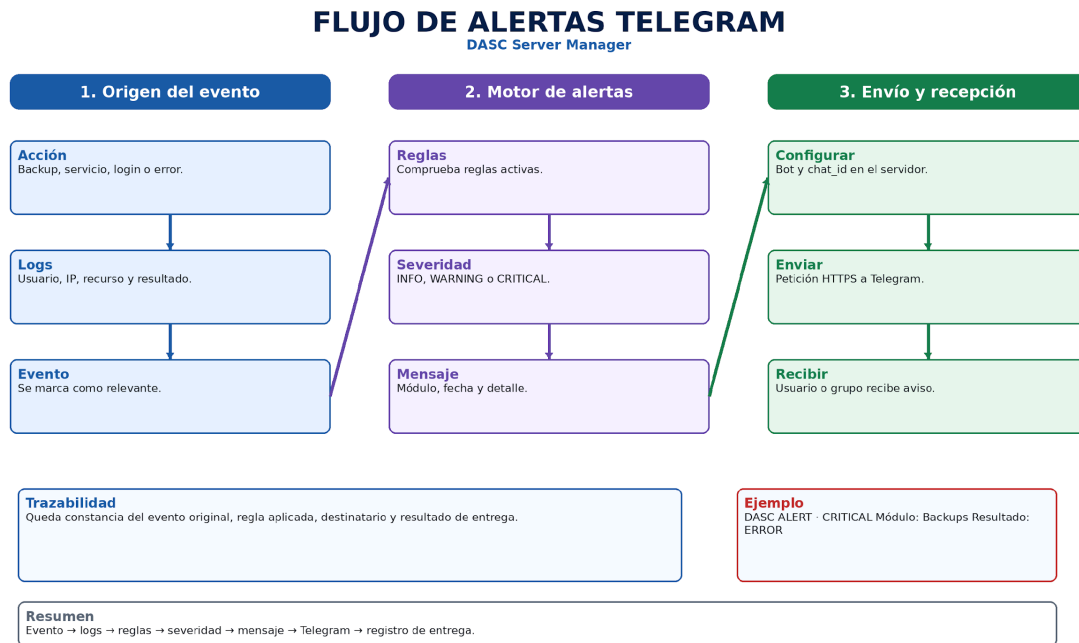


Figura 11. Flujo de alertas Telegram.

El módulo de alertas permite configurar destinatarios individuales o grupos de Telegram, enviar pruebas y detectar chats recientes mediante `getUpdates`. Internamente se usa una base SQLite para no depender de otra base externa. Las reglas de alerta pueden activarse o desactivarse desde el panel.

- Canal principal: Telegram.
- Destinatarios: usuarios o grupos identificados por `chat_id`.
- Reglas: backup.ok, backup.error, restore, delete y otros eventos del panel.
- Entregas: se registra si el envío fue OK o ERROR.
- Manual de ayuda: guía PDF accesible desde `static/docs`.

Evidencia en anexos: Panel de alertas con destinatarios y prueba enviada correctamente, con prueba enviada correctamente. Ver Annex D de capturas.

5.12 Instaladores y despliegue

El proyecto incluye instaladores por rol. Esta decisión evita tener que ejecutar muchos comandos manuales y mejora la reproducibilidad. El orden recomendado es instalar primero la base de datos, después backups/servicios y finalmente la API.

Script	Dónde se ejecuta	Responsabilidad
deploy/db/install_db.sh	192.168.60.20	Instala MariaDB, SSH, binlogs, base employees, usuarios dasc_backup y dasc_restore.
deploy/backup-services/install_backup_services.sh	192.168.60.30	Instala SSH, cron, clientes MySQL, scripts, .my.cnf, .my_restore.cnf y sudoers.
deploy/api/install_dasc_api.sh	192.168.60.10	Copia la API a /opt/dasc/api, crea venv, instala dependencias, configura admin y SSH aislado.
uninstall_*.sh	Cada rol	Limpia servicios, paquetes y archivos generados.
deploy/api/cacti/install_cacti.sh	API o máquina de monitorización	Instala Cacti mínimo cuando se usa versión no Docker.
Evidencia en anexos: Ejecución final de install_dasc_api.sh mostrando servicio activo y URL de acceso. Ver Annex D de capturas.		

5.13 Implementación Docker

La versión Docker se ha preparado como alternativa de demostración. Define contenedores dasc-web, mysql-db, backup-services, cacti y cacti-db. La red dasc_lab_net asigna IPs equivalentes al laboratorio, lo que permite mantener la lógica de la API y simular las tres máquinas desde Docker Compose.

```
docker compose up -d --build
docker compose ps
docker exec mysql-db mariadb -uroot -prootpass -e "SHOW MASTER STATUS;"
Accesos:
Panel: http://localhost:8080
Cacti: http://localhost:8081/cacti/
```

Evidencia en anexos: docker compose ps con todos los servicios Up. Ver Annex D de capturas.

5.14 Manual resumido de instalación por máquinas

El despliegue no debe depender de ejecutar comandos sueltos recordados de memoria. Por eso se han preparado instaladores por rol. La memoria final debe explicar el orden y dejar los comandos básicos de validación, mientras que los detalles largos pueden quedar en anexos o README.

5.14.1 Instalación de la máquina de base de datos

```
cd ~/dasc-server-manager/deploy/db
chmod +x install_db.sh uninstall_db.sh
sudo ./install_db.sh
systemctl status mariadb --no-pager
```

```
mariadb -e "SHOW MASTER STATUS;"
mariadb -e "SHOW BINARY LOGS;"
```

Esta instalación prepara MariaDB para aceptar conexiones desde el servidor de backups, crea usuarios separados para backup y restauración, activa binlogs y deja un usuario SSH para la terminal remota del panel.

5.14.2 Instalación de la máquina de backups y servicios

```
cd ~/dasc-server-manager/deploy/backup-services
chmod +x install_backup_services.sh uninstall_backup_services.sh
sudo ./install_backup_services.sh
ls -l /usr/local/bin/backups_api.sh /usr/local/bin/restore_api.sh
/usr/local/bin/servicios_api.sh
sudo -u dasc mysql --defaults-extra-file=/home/dasc/.my.cnf --protocol=tcp -h 192.168.60.20
-e "SHOW DATABASES;"
```

Esta máquina almacena las copias y ejecuta los scripts. También tiene cron activo para poder programar automatizaciones de backups desde el panel.

5.14.3 Instalación de la máquina API

```
cd ~/dasc-server-manager/deploy/api
chmod +x install_dasc_api.sh update_dasc_api.sh uninstall_dasc_api.sh
sudo ./install_dasc_api.sh
systemctl status dasc-api --no-pager
curl -I http://127.0.0.1:8000
```

Durante la instalación de la API se pide la contraseña del administrador del panel y se configura una clave SSH propia de DASC. Esta clave se guarda dentro de `/opt/dasc/api/.ssh` y no utiliza la clave personal del usuario del sistema.

5.15 Implementación de Cacti y monitorización

Cacti se incluye como herramienta de monitorización para complementar la auditoría del panel. Mientras que `/logs` muestra acciones del usuario y eventos del backend, Cacti permite justificar una capa de supervisión técnica sobre la infraestructura. En la versión Docker se ejecuta con `cacti` y `cacti-db`; en la versión por máquinas existe instalador mínimo dentro de `deploy/api/cacti`.

- Cacti aporta una vista de monitorización separada del panel principal.
- La integración desde DASC se realiza mediante un enlace configurado por `CACTI_URL`.
- En Docker se publica normalmente en `http://localhost:8081/cacti/`.
- En la documentación se debe incluir captura de acceso a Cacti y, si es posible, una gráfica o panel inicial.

- Si Cacti no está totalmente personalizado, se debe explicar como herramienta de monitorización integrada, no como desarrollo propio.

Evidencia en anexos: Pantalla de Cacti funcionando desde el botón Abrir Cacti del panel de logs. Ver Annex D de capturas.

6. Test

6.1 Estrategia de pruebas

El testing del proyecto se ha planteado como pruebas funcionales manuales y pruebas técnicas desde terminal. Al ser un proyecto de administración de sistemas, no basta con comprobar que la interfaz carga: también hay que validar que los scripts remotos, los servicios, la base de datos y los archivos generados existen realmente en las máquinas correspondientes.

- Pruebas de despliegue: comprobar que los instaladores finalizan sin errores.
- Pruebas de conectividad: ping, SSH y conexión al puerto 3306.
- Pruebas de autenticación: login correcto, login incorrecto y logout.
- Pruebas de autorización: usuarios con permisos limitados.
- Pruebas de backups: full, incremental, differential y validación de historial.
- Pruebas de restauración: preview, confirmación y copia preventiva.
- Pruebas de eliminación: preview de dependencias y confirmación.
- Pruebas de servicios: listado y reinicio de servicio no crítico.
- Pruebas de logs: comparación entre panel y consulta SQL directa.
- Pruebas de alertas: prueba Telegram y registro de entrega.

6.2 Tabla de pruebas principales

ID	Prueba	Procedimiento	Resultado esperado
T01	Instalación DB	Ejecutar install_db.sh	MariaDB activo, binlogs activos, usuarios creados
T02	Instalación Backups	Ejecutar install_backup_services.sh	SSH, cron, scripts y mysqlbinlog disponibles
T03	Instalación API	Ejecutar install_dasc_api.sh	Servicio dasc-api activo en 8000
T04	Login correcto	Entrar con admin	Dashboard completo visible

ID	Prueba	Procedimiento	Resultado esperado
T05	Login incorrecto	Credenciales falsas	Mensaje de error y log de fallo
T06	Permisos	Usuario solo logs	No ve backups ni servicios
T07	Backup full	Crear full employees	Archivo .sql.gz e ID en history.tsv
T08	Backup incremental	Crear incremental tras full	Archivo binlog y base_id correcto
T09	Backup diferencial	Crear diferencial tras full	Archivo binlog y base_id del full
T10	Restauración preview	Seleccionar ID	Cadena calculada sin ejecutar aún
T11	Restauración confirm	Confirmar SI	Copia preventiva y restauración OK
T12	Eliminación	Eliminar full con dependientes	Advertencia de cascada
T13	Servicios	Restart cron.service	Servicio activo después de acción
T14	Logs	Ejecutar acciones y abrir /logs	Eventos recientes visibles
T15	Alertas	Enviar prueba Telegram	Entrega OK registrada
T16	Docker	docker compose up -d --build	Contenedores Up

6.3 Comandos de validación recomendados

```
# API
systemctl status dasc-api --no-pager
curl -I http://127.0.0.1:8000
journalctl -u dasc-api -n 50 --no-pager

# Base de datos
systemctl status mariadb --no-pager
ss -lntp | grep 3306
mariadb -e "SHOW MASTER STATUS;"
mariadb -e "SHOW BINARY LOGS;"

# Backups + servicios
systemctl status ssh --no-pager
systemctl status cron --no-pager
ls -lh /home/dasc/backups
cat /home/dasc/backups/.dasc/history.tsv
/usr/local/bin/servicios_api.sh list | head

# Desde API hacia backups
ssh -i /opt/dasc/api/.ssh/id_rsa dasc -o
UserKnownHostsFile=/opt/dasc/api/.ssh/known_hosts dasc dasc@192.168.60.30 hostname
```

Evidencia en anexos: Comandos de validación con resultados correctos. Ver Annex D de capturas.

6.4 Incidencias detectadas durante las pruebas

Incidencia	Causa	Solución / aprendizaje
Host key verification failed	Reinstalaciones cambiaban la huella SSH.	Crear known_hosts aislado de DASC y registrar host keys con ssh-keyscan.
Fallos al ejecutar scripts por SSH	Clave no autorizada o usuario dasc sin permisos.	Instalador copia clave con sshpass y valida BatchMode.
Backups incrementales sin base	No existía copia full previa.	Validar historial y mostrar error claro.
mysqlbinlog no disponible	Cliente MySQL incompleto.	Instalar mysql-client-8.0 y mysql-server-core-8.0 en backup-services.

6.5 Pruebas específicas de backups

Las pruebas de backups deben demostrar tres cosas: que se crea el fichero, que el historial se actualiza y que las dependencias son correctas. Para incrementales y diferenciales no basta con ver un archivo; hay que revisar base_id y posiciones de binlog.

Paso	Acción	Validación
1	Crear backup full desde panel.	history.tsv contiene una línea type=full sin base_id.
2	Insertar o modificar datos en employees.	La base cambia antes de incremental/differential.
3	Crear backup incremental.	history.tsv contiene type=incremental y base_id del último backup.
4	Modificar datos de nuevo.	Hay nuevos eventos en binlog.
5	Crear backup differential.	base_id apunta al último full, no necesariamente al último incremental.
6	Probar restauración preview.	La cadena muestra full + incrementales/diferencial necesario.
7	Ejecutar restauración confirmada.	Se crea copia preventiva pre-restore y resultado OK.

```
# Insertar cambio de prueba en la DB para generar binlog
mariadb employees -e "INSERT INTO empleados_demo(nombre) VALUES ('prueba-binlog-final');"
```

```
# Validar historial en servidor de backups
cat /home/dasc/backups/.dasc/history.tsv
```

```
# Validar ficheros generados
ls -lh /home/dasc/backups
ls -lh /home/dasc/backups/pre-restore 2>/dev/null || true
```

6.6 Pruebas específicas de alertas

Las alertas deben probarse con un bot controlado y con un token nuevo. La prueba correcta es enseñar el panel con destinatario configurado, ejecutar una prueba y mostrar el mensaje recibido en Telegram.

1. Crear o regenerar token desde BotFather.
2. Configurar el token del bot en el servidor.
3. Reiniciar dasc-api.
4. Entrar en /alertas.
5. Detectar chat o añadir chat_id manualmente.
6. Marcar destinatario por defecto.
7. Enviar prueba de alerta.
8. Comprobar que aparece entrega OK y que Telegram recibe el mensaje.

Evidencia en anexos: Prueba de Telegram enviada correctamente sin mostrar token. Ver Annex D de capturas.

7. Resultado

7.1 Resultado final del producto

El resultado actual es un panel funcional que centraliza operaciones de administración sobre un entorno distribuido. Aunque todavía quedan pendientes concretos de seguridad, el proyecto ya permite demostrar una solución realista: autenticación, control de permisos, ejecución de copias, restauración, logs, alertas y gestión de servicios.

Módulo	Estado	Evidencia a incluir
Login y sesiones	Funcional	Captura login OK y login incorrecto.
Usuarios y permisos	Funcional	Captura usuario limitado y menú reducido.
Dashboard	Funcional	Captura del panel principal.
Backups	Funcional	Captura formulario + archivo generado.
Restauración	Funcional / validar	Captura preview + confirmación.
Descarga	Funcional / validar	Captura descarga del backup.
Eliminación	Funcional / validar	Captura aviso de dependencias.
Servicios	Funcional	Captura reinicio de cron o ssh.
Logs	Funcional	Captura tabla de eventos.
Alertas	Funcional	Captura prueba Telegram sin mostrar token.
Terminal	Funcional	Captura comando hostname/ip a.
Docker	Funcional / demo	Captura docker compose ps.

7.2 Guía de uso para el usuario

9. Abrir el navegador y acceder al panel DASC Server Manager.
10. Iniciar sesión con un usuario válido.
11. Revisar las tarjetas disponibles en el dashboard según permisos.
12. Entrar en Copias para ejecutar o revisar backups.
13. Entrar en Logs para ver auditoría y acceder a Cacti.
14. Entrar en Servicios para iniciar, parar o reiniciar servicios permitidos.
15. Entrar en Alertas para configurar Telegram y probar entregas.
16. Entrar en Terminal solo si el usuario tiene permisos técnicos.
17. Cerrar sesión al finalizar la administración.

Evidencia en anexos: Secuencia de uso: login, dashboard, backups, logs, servicios y logout. Ver Annex D de capturas.

7.3 Control de calidad del producto

Para considerar el producto listo para la presentación final, se deben cumplir tres condiciones: las funcionalidades principales deben poder demostrarse sin comandos manuales no documentados, los errores deben mostrarse de forma controlada y no deben quedar secretos reales en el repositorio ni en las capturas.

- El panel debe arrancar con systemd sin ejecutar uvicorn manualmente.
- Los scripts remotos deben ejecutarse por SSH con clave y sin pedir contraseña al panel.
- Los backups generados deben verse tanto en la web como en el sistema de archivos.
- Los eventos del panel deben aparecer en /logs y en la base dasc_logs.
- Las alertas deben poder probarse con un chat propio controlado por el grupo.

7.4 Manual de usuario detallado por módulos

7.4.1 Uso del módulo de copias

El usuario con permiso de copias accede a /backups y selecciona el tipo de copia. Para una copia completa se recomienda usar un nombre con fecha, por ejemplo backup-YYYYMMDD-HHMM.sql. Para incremental y diferencial, el sistema usa el historial para determinar la base. Si no existe una copia full previa, la operación debe fallar con un mensaje claro.

7.4.2 Uso del módulo de restauración

La restauración no se ejecuta directamente al primer clic. Primero se muestra una previsualización de la cadena de copias necesaria. El usuario debe confirmar explícitamente con SI. Esta doble confirmación evita restauraciones accidentales y permite comprobar si la cadena afecta a la base de datos esperada.

7.4.3 Uso del módulo de servicios

El módulo de servicios muestra unidades systemd remotas. Para la demostración se recomienda usar un servicio no crítico como cron.service. No se recomienda parar ssh.service durante la defensa, porque podría cortar el acceso remoto necesario para seguir demostrando el proyecto.

7.4.4 Uso del módulo de logs

La vista de logs sirve para comprobar que el panel no solo ejecuta acciones, sino que también las registra. Después de hacer login, ejecutar un backup o reiniciar un servicio, se debe volver a /logs y enseñar la nueva línea generada. Esta es una prueba clara de auditoría.

7.4.5 Uso del módulo de alertas

El módulo de alertas permite añadir destinatarios, marcar un chat por defecto y enviar una prueba. Para la presentación se recomienda tener un grupo o chat preparado previamente, con token nuevo y sin mostrar secretos en pantalla.

7.4.6 Uso del módulo de terminal

La terminal se debe presentar como herramienta avanzada. Una buena demostración consiste en ejecutar hostname, ip a o systemctl status sobre una máquina. No se deben ejecutar comandos destructivos ni mostrar archivos con contraseñas.

7.5 Guion breve de demostración final

Para la presentación se recomienda seguir un guion estable y no improvisar demasiado. El objetivo es enseñar primero la arquitectura y después demostrar acciones funcionales en orden lógico.

18. Mostrar diagrama de arquitectura y explicar las tres máquinas.
19. Entrar al panel con usuario administrador.
20. Mostrar dashboard y permisos.
21. Crear o mostrar usuario limitado.
22. Ejecutar un backup full ya preparado o mostrar historial con full/incremental/differential.
23. Mostrar archivo generado en servidor de backups.
24. Mostrar logs generados por la acción.
25. Reiniciar cron.service desde Servicios.
26. Enviar alerta Telegram de prueba.

8. Pla de sostenibilitat

8.1 Sostenibilitat ambiental

DASC Server Manager se ha desarrollado en un entorno académico virtualizado, sin necesidad de comprar hardware físico adicional. El uso de máquinas virtuales y contenedores permite reutilizar recursos existentes y reducir el impacto ambiental asociado a la adquisición de equipamiento nuevo.

- No se ha adquirido hardware físico específico para el MVP.
- Se aprovechan máquinas virtuales y software libre.
- La versión Docker permite hacer demostraciones sin mantener servidores físicos encendidos permanentemente.
- Los backups incrementales y diferenciales reducen almacenamiento frente a copias completas constantes.
- La retención configurada evita crecimiento infinito de ficheros de backup.

8.2 Sostenibilidad económica

El proyecto se apoya en tecnologías libres o gratuitas: Ubuntu, Python, FastAPI, MariaDB, Cacti, Bash y Docker. Esto reduce el coste inicial y facilita que pequeñas empresas puedan adoptar una solución similar sin licencias elevadas.

Elemento	Coste de licencia	Justificación
Ubuntu Server	0 euros	Sistema operativo libre y ampliamente documentado.
Python/FastAPI	0 euros	Framework y lenguaje sin coste de licencia.
MariaDB	0 euros	Base de datos libre compatible con el enfoque del proyecto.
Cacti	0 euros	Herramienta de monitorización libre.
Docker	0 euros en uso académico/local	Permite reproducir el entorno fácilmente.

8.3 Sostenibilidad social y legal

El sistema favorece la transparencia porque registra acciones y accesos. En un entorno empresarial, saber quién ejecuta una copia, quién restaura datos o quién reinicia un servicio ayuda a mejorar la responsabilidad interna y la trazabilidad.

- Registro de accesos y operaciones administrativas.
- Control de permisos por usuario.
- Reducción de dependencia de terceros externos.
- Mayor soberanía tecnológica sobre datos y copias.
- Mejor capacidad de auditoría ante incidencias.

8.4 Relación con ODS

ODS	Relación con el proyecto
ODS 9 - Industria, innovación e infraestructuras	El proyecto fomenta infraestructura tecnológica propia, segura y automatizada.
ODS 12 - Producción y consumo responsables	Optimiza almacenamiento mediante copias incrementales/diferenciales y retención.
ODS 8 - Trabajo decente y crecimiento económico	Facilita a PYMES una herramienta local de bajo coste y mejora la continuidad de negocio.

9. Valoració funcional

9.1 Objetivos alcanzados

La valoración funcional es positiva porque el proyecto ha pasado de un MVP centrado en backups completos y servicios básicos a una solución más amplia. El panel ya permite una demostración completa de administración de servidores con módulos diferenciados.

Objetivo inicial	Resultado actual	Valoración
Panel web local	Implementado con FastAPI y Uvicorn.	Alcanzado.
Backups completos	Implementado y ampliado a incremental/differential mediante binlogs.	Superado si las pruebas finales son correctas.
Historial de backups	Implementado mediante history.tsv y vista web.	Alcanzado.
Restauración	Implementada con preview, confirmación y copia preventiva.	Alcanzado / validar con captura.
Descarga y eliminación	Implementadas con validación de ruta y dependencias.	Alcanzado / validar con captura.
Gestión de servicios	Implementada vía SSH y systemctl.	Alcanzado.
Logs	Implementados en MariaDB y auth_logs.json.	Alcanzado.
Alertas	Implementadas con Telegram y SQLite.	Alcanzado.

9.2 Limitaciones actuales

- La terminal remota es potente y debe limitarse a usuarios administradores o técnicos.
- La monitorización con Cacti debe demostrarse con capturas reales del entorno definitivo.
- La versión Docker es útil para demo, pero la memoria ASIX debe mantener como principal el diseño por máquinas.

9.3 Mejoras futuras

- Añadir HTTPS real con certificado interno o reverse proxy.
- Añadir doble factor para usuarios administradores.
- Añadir cifrado de backups en reposo.
- Añadir gestión de rotación de tokens y secretos desde instalador.
- Añadir pruebas automáticas del backend.
- Añadir dashboard de métricas más completo.

- Preparar roles de usuario más granulares y registro de acciones firmadas.

10. Valoració tècnica

10.1 Herramientas de desarrollo

El desarrollo se ha realizado combinando herramientas de programación, sistemas y documentación. La elección de FastAPI y Bash ha permitido avanzar rápido sin perder contacto con la administración real de Linux.

Herramienta	Uso	Valoración
GitHub	Repositorio público del proyecto.	Útil para control de versiones, pero exige cuidado con secretos.
Google Drive / Docs	Documentación y entregas.	Adecuado para trabajo colaborativo.
Ubuntu / terminal	Pruebas de despliegue.	Imprescindible para validar systemd, SSH y MariaDB.
Docker Desktop / Compose	Demo reproducible.	Ayuda a presentar y probar sin recrear todas las VMs.
Cacti	Monitorización.	Correcto para justificar supervisión, aunque requiere configuración cuidadosa.
Packet Tracer / draw.io	Diagramas de red.	Necesario para la parte ASIX de diseño.

10.2 Valoración de tecnologías

FastAPI ha sido una buena elección porque permite servir páginas HTML y endpoints de API desde el mismo backend. MariaDB encaja con la necesidad de demostrar backups reales mediante mysqldump y binlogs. Bash sigue siendo adecuado para tareas de sistema porque los instaladores y scripts administrativos son más transparentes y fáciles de auditar en un proyecto ASIX.

La principal dificultad técnica ha estado en la integración entre capas: permisos SSH, known_hosts, sudoers, credenciales de MariaDB, binlogs, systemd y renderizado del panel. Estas dificultades son precisamente las que aportan valor al proyecto, porque demuestran administración real de sistemas y no solo una interfaz simulada.

10.3 Alternativas técnicas

Elemento actual	Alternativa	Motivo para considerarla
FastAPI + Jinja2	Django	Django aporta ORM y administración integrada, pero sería más pesado.

Elemento actual	Alternativa	Motivo para considerarla
Bash scripts	Ansible	Ansible mejoraría automatización y repetibilidad en varios servidores.
Telegram	Email / Slack / Teams	Canales más empresariales para producción.
users.json	Base de datos de usuarios	Mejor escalabilidad y control de auditoría.
Cacti	Grafana + Prometheus	Dashboard moderno y flexible, aunque más complejo.
SSH directo	Agente/API por servidor	Mayor control, pero requiere desplegar componentes adicionales.

10.4 Seguridad técnica

La seguridad técnica del proyecto se apoya en varias capas: autenticación, permisos por módulo, separación entre servidores, acceso remoto mediante SSH, usuarios específicos de base de datos y registro de acciones relevantes.

Riesgo	Impacto	Control aplicado o pendiente
Terminal remota	Ejecución indebida de comandos.	Permiso específico, logs y límites de comando.
SSH remoto	Acceso no autorizado si se filtra clave.	Clave aislada, permisos 600 y known_hosts controlado.
Backups descargables	Exfiltración de datos.	Validar ruta y limitar permisos.

10.5 Incidencias reales y decisiones tomadas

Una parte importante de la valoración técnica es explicar problemas reales. No todos los errores son negativos; algunos han servido para mejorar la arquitectura y justificar decisiones de seguridad.

Problema real	Qué ocurrió	Decisión tomada
Docker daemon no activo	docker compose no podía descargar/levantar imágenes si Docker Desktop no estaba iniciado.	Documentar comprobación previa de Docker y usar docker compose ps/logs.
Cacti con configuración delicada	Cacti requiere base, rutas, sesiones y permisos concretos.	Separar cacti y cacti-db en Docker y preparar instalador/entrypoint.
Host keys SSH cambiantes	Las reinstalaciones provocaban errores de verificación.	Usar known_hosts aislado y ssh-keyscan en instalador/update.
mysqlbinlog y MariaDB	El cliente necesario no siempre estaba instalado.	Instalar herramientas MySQL necesarias en backup-services.

Estas incidencias refuerzan el valor del proyecto porque muestran aprendizaje práctico: no solo se ha creado una aplicación, sino que se han resuelto problemas habituales de despliegue, redes, permisos, servicios y seguridad.

11. Valoració de la gestió

11.1 Metodología de trabajo

La gestión del proyecto ha combinado planificación por tareas, reuniones de seguimiento y desarrollo incremental. El equipo ha ido ajustando el alcance a medida que aparecían dificultades técnicas, manteniendo como prioridad que el producto final fuera demostrable y no solo teórico.

- Trabajo por módulos: API, frontend, backups, base de datos, servicios, logs, alertas y Docker.
- Revisión continua de errores mediante terminal, journalctl, logs de Docker y pruebas desde navegador.
- Uso de GitHub como repositorio central del código.
- Uso de documentación compartida para memorias, guías y capturas.
- Priorización de funcionalidades visibles en la presentación final.

11.2 Planificación y reajustes

La planificación inicial se centraba en un MVP con backups completos y una interfaz básica. Con el avance del proyecto se añadieron funcionalidades más avanzadas: roles, alertas, terminal, automatización de backups, restauración, eliminación con dependencias y Docker. Esto obligó a reajustar la documentación y la presentación para explicar correctamente la evolución.

Fase	Trabajo principal	Resultado
Primera fase	Idea, objetivos, viabilidad y planificación.	Definición de DASC Server Manager.
MVP	Panel, backups básicos, servicios y logs iniciales.	Prototipo funcional.
Evolución final	Permisos, alertas, terminal, restauración, binlogs y Docker.	Producto casi completo.

11.3 Herramientas de colaboración

Las herramientas utilizadas han permitido repartir trabajo y centralizar evidencias. GitHub ha sido especialmente útil para mantener scripts e instaladores, aunque también ha demostrado la necesidad de aplicar buenas prácticas de secretos. Google Drive/Docs ha servido para la documentación y el intercambio de materiales.

11.4 Reparto de tareas

El reparto se ha organizado por áreas: frontend y diseño del panel, backend/API, scripts de backups y servicios, base de datos, red/infraestructura, Docker, documentación y pruebas. En la fase final, muchas tareas se han cruzado porque los módulos ya están integrados.

Área	Responsabilidades
Frontend	Pantallas HTML, CSS, navegación, experiencia de usuario y capturas.
Backend/API	Rutas FastAPI, sesiones, permisos, logs, terminal y alertas.
Backups	backups_api.sh, restore_api.sh, historial, binlogs y automatización.
Base de datos	MariaDB, usuarios backup/restore, binlogs, logs y pruebas SQL.
Servicios	servicios_api.sh, sudoers y systemctl remoto.
Infraestructura	VMs, IPs, SSH, Cacti y red.
Docker	docker-compose.yml, contenedores y guía de demostración.
Documentación	Memoria final, capturas, diagramas y presentación.

11.5 Documentación técnica de administración

Para mantener el sistema, un administrador debe conocer el orden de instalación, la ubicación de los archivos, los servicios systemd y las rutas de los logs y backups. Esta información debe estar presente tanto en la memoria como en manuales anexos.

Elemento	Ruta / comando
Instalación API	/opt/dasc/api
Servicio API	systemctl status dasc-api
Clave SSH DASC	/opt/dasc/api/.ssh/id_rsa_dasc
Known hosts DASC	/opt/dasc/api/.ssh/known_hosts_dasc
Backups	/home/dasc/backups
Historial backups	/home/dasc/backups/.dasc/history.tsv
Script backups	/usr/local/bin/backups_api.sh
Script restore	/usr/local/bin/restore_api.sh
Script servicios	/usr/local/bin/servicios_api.sh
Logs API	journalctl -u dasc-api
Logs panel	Base dasc_logs, tabla eventos

11.6 Retrospectiva final del equipo

La evolución del proyecto demuestra que el alcance inicial se fue ampliando de forma progresiva. Al principio el objetivo principal era realizar backups y mostrar una interfaz. En la

fase final, el producto incluye más módulos y se aproxima a una herramienta de administración completa. Esto ha aumentado la complejidad, pero también ha dado más contenido técnico para defender en la presentación.

- Lo que mejor ha funcionado: dividir por módulos y validar con scripts independientes antes de integrarlos en el panel.
- Lo más difícil: coordinar credenciales, permisos, SSH, MariaDB, systemd y Docker sin romper el despliegue.
- Lo más importante aprendido: un proyecto de sistemas necesita tanta atención a configuración y seguridad como a código.
- Lo que queda como mejora: automatizar pruebas, añadir HTTPS, endurecer terminal y preparar una instalación empresarial más guiada.

11.7 Criterios de cierre de documentación

Antes de entregar, la documentación debe revisarse con criterios de formato y contenido. La guía oficial exige Calibri 12, interlineado 1,5, márgenes estándar, paginación y entre 40 y 60 páginas de contenido sin contar portada, índice ni anexos. Por tanto, después de insertar capturas reales conviene exportar a PDF y revisar que no haya tablas cortadas ni numeración incorrecta.

Elemento	Revisión final
Numeración	Comprobar que los apartados siguen el orden obligatorio.
Índice	Actualizar índice automático en Google Docs.
Capturas	Añadir capturas reales y ocultar secretos.
Diagramas	Insertar red lógica, física, servicios y seguridad.
Páginas	Mantener entre 40 y 60 páginas de contenido.
Anexos	Mover comandos largos y capturas repetidas a anexos.
Bibliografía	Revisar formato APA7.
Seguridad	No incluir tokens, contraseñas ni claves reales.

11.8 Checklist de entrega final

Para evitar errores de última hora, el equipo debe realizar una revisión final siguiendo una lista cerrada. Esta lista mezcla requisitos técnicos, requisitos de documentación y requisitos de seguridad.

Bloque	Comprobación	Estado recomendado
Formato	Calibri 12, interlineado 1,5, márgenes 2,5 cm y paginación.	Revisado en Google Docs y PDF.
Índice	Tabla de contenidos automática con enlaces clicables.	Actualizar justo antes de entregar.
Capturas	Todas las capturas tienen título y explicación breve.	Añadir solo capturas necesarias en memoria principal.
Diagramas	Red lógica, red física, arquitectura, backups, logs y seguridad.	Insertar versiones finales legibles.
Backups	Full, incremental, differential, restore, download y delete probados.	Capturas finales añadidas.
Logs	Eventos visibles tanto en panel como en base de datos.	Validado con acción reciente.
Alertas	Telegram probado con token nuevo.	No mostrar token.
Repositorio	README actualizado y sin secretos reales.	Revisión final con grep.
Presentación	Demo ensayada en orden y sin depender de improvisación.	Preparada con datos de prueba.

12. Bibliografía i Webgrafia

Las fuentes deben revisarse y formatearse en APA7 antes de la entrega definitiva. A continuación se incluye una propuesta base.

FastAPI. (s. f.). FastAPI documentation. <https://fastapi.tiangolo.com/>

Uvicorn. (s. f.). Uvicorn documentation. <https://www.uvicorn.org/>

MariaDB Foundation. (s. f.). MariaDB Server documentation.
<https://mariadb.org/documentation/>

Docker. (s. f.). Docker Compose documentation. <https://docs.docker.com/compose/>

Cacti. (s. f.). Cacti documentation. <https://www.cacti.net/>

Python Software Foundation. (s. f.). Python documentation. <https://docs.python.org/>

Telegram. (s. f.). Telegram Bot API. <https://core.telegram.org/bots/api>

Institut Manuel Vázquez Montalbán. (2026). Guia de l'entrega final del projecte. Documento interno de proyecto.

Repositorio del proyecto DASC Server Manager.

<https://github.com/scolmena10/dasc-server-manager>

Annexos

Annex A. Lista de capturas recomendadas

Los anexos no deberían sustituir la explicación principal, pero sirven para incluir evidencias extensas sin sobrecargar las 60 páginas de memoria. Las capturas siguientes se recomiendan para completar la documentación final.

ID	Captura	Cómo obtenerla	Dónde usarla
A01	IPs de las tres máquinas	hostname, ip a	Diseño / Test
A02	Conectividad API -> DB y API -> Backups	ping y nc	Test
A03	Instalación MariaDB	sudo ./install_db.sh	Implementación
A04	MariaDB y binlogs activos	SHOW MASTER STATUS; SHOW BINARY LOGS;	Test
A05	Instalación backup-services	sudo ./install_backup_services.sh	Implementación
A06	Scripts instalados	ls -l /usr/local/bin/*.sh	Implementación
A07	Instalación API	sudo ./install_dasc_api.sh	Implementación
A08	Servicio dasc-api	systemctl status dasc-api	Test
A09	Login	Navegador /login	Resultado
A10	Dashboard admin	Usuario admin	Resultado
A11	Usuarios/permisos	/admin/usuarios	Resultado
A12	Backup full OK	/backups	Test
A13	Backup incremental/diferencial OK	/backups	Test
A14	Historial de backups	history.tsv o panel	Test
A15	Restauración preview/confirm	/backups	Test
A16	Descarga backup	/backups/download	Resultado
A17	Eliminación con dependencias	/backups/delete/preview	Test
A18	Logs del panel	/logs	Resultado
A19	Servicios	/servicios	Resultado
A20	Alertas Telegram	/alertas	Resultado
A21	Terminal remota	/terminal	Resultado
A22	Docker	docker compose ps	Anexo / Demo

Annex B. Diagramas obligatorios y recomendados

Diagrama	Obligatorio	Descripción
Diagrama lógico de red	Sí	IPs, subred, tráfico HTTP/SSH/MariaDB.

Diagrama	Obligatorio	Descripción
Diagrama físico de red	Sí	VMs, switch/red interna y equipo administrador.
Tabla de direccionamiento	Sí	Equipo, rol, IP y servicios.
Esquema de servicios	Sí	Puertos, origen, destino y función.
Arquitectura de componentes	Recomendado	FastAPI, templates, static, users, logs, scripts.
Flujo de backups	Recomendado	Formulario -> API -> SSH -> script -> MariaDB -> archivo.
Flujo de restauración	Recomendado	ID objetivo -> cadena desde full -> copia preventiva -> apply SQL.
Flujo de logs y alertas	Recomendado	Middleware -> eventos -> /logs -> Telegram.
Diagrama de seguridad	Recomendado	Sesión, permisos, SSH, DB users, secretos.
Arquitectura Docker	Opcional	Contenedores, redes y volúmenes.

Annex C. Plan de cierre funcional antes de entregar

Este anexo resume las comprobaciones finales recomendadas antes de la entrega. Se centra en validar el funcionamiento real del producto, las capturas y la demostración, sin añadir funcionalidades nuevas de última hora.

- Actualizar el índice del documento después de insertar capturas y diagramas.
- Comprobar que las capturas principales aparecen en anexos y que no se cortan.
- Validar instalación de DB, backup-services y API con los scripts finales.
- Probar login correcto, login incorrecto y usuario con permisos limitados.
- Ejecutar backup full, incremental y diferencial y revisar el historial.
- Probar descarga, restauración preview y eliminación con confirmación.
- Reiniciar un servicio no crítico desde el panel de servicios.
- Comprobar que los eventos aparecen en la vista de logs.
- Enviar una alerta de prueba a Telegram y guardar evidencia del resultado.
- Ejecutar docker compose ps si se usa la versión Docker en la demo.
- Exportar a PDF y revisar que no haya imágenes cortadas ni tablas partidas.

Annex D. Capturas del proyecto

Las siguientes capturas se incluyen en anexos para no sobrecargar la memoria principal. Están ordenadas por número de archivo y sirven como evidencias de instalación, pruebas, interfaz y resultados.

Captura D.1. 02 conectividad api db backups

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp1s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 52:54:00:16:3d:36 brd ff:ff:ff:ff:ff:ff
   inet 192.168.120.187/22 brd 192.168.123.255 scope global dynamic noprefixroute enp1s0
       valid_lft 3270sec preferred_lft 3270sec
   inet6 fe80::3967:8894:fb49:486f/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: enp2s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 52:54:00:57:c4:83 brd ff:ff:ff:ff:ff:ff
   inet 192.168.60.30/24 brd 192.168.60.255 scope global noprefixroute enp2s0
       valid_lft forever preferred_lft forever
   inet6 fe80::ac48:4a8:50f1:4953/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Captura D.2. 04 install db ok

```
=====
Base de datos instalada correctamente
DB_NAME=employees
TEST_TABLE=empleados_demo
BACKUP_USER=dasc_backup
BACKUP_ALLOWED_HOST=192.168.60.30
Binary logs: /var/log/mysql/dasc-bin
Puerto esperado: 3306
=====
```

Captura D.3. 05 mariadb status puerto

```
alume@equino-lizard:~$ systemctl status mariadb --no-pager
● mariadb.service - MariaDB 10.6.23 database server
   Loaded: loaded (/lib/systemd/system/mariadb.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-05-15 16:14:31 CEST; 1min 12s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 674 ExecStartPre=/usr/bin/install -m 755 -o mysql -g root -d /var/run/mysql (code=exited, status=0/SUCCESS)
   Process: 691 ExecStartPre=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=0/SUCCESS)
   Process: 695 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && VAR= || VAR= /usr/bin/galera_recovery ; [ $? -eq 0 ] && systemctl set-environment _WSREP_START_POSITION=$VAR || ex
   Process: 844 ExecStartPost=/bin/sh -c systemctl unset-environment _WSREP_START_POSITION (code=exited, status=0/SUCCESS)
   Process: 850 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/SUCCESS)
  Main PID: 755 (mariadbd)
    Status: "Taking your SQL requests now..."
     Tasks: 10 (limit: 30347)
   Memory: 91.8M
      CPU: 429ms
   CGroup: /system.slice/mariadb.service
           └─755 /usr/sbin/mariadbd
```

Captura D.4. 06 binlogs show master status

```
alume@db-santino:~$ sudo mariadb -e "SHOW MASTER STATUS;"
+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+-----+-----+-----+-----+
| dasc-bin.000005 |      341 |               |                   |
+-----+-----+-----+-----+
alume@db-santino:~$ sudo mariadb -e "SHOW VARIABLES LIKE 'log_bin';"
+-----+-----+
| Variable_name | Value |
+-----+-----+
| log_bin       | ON    |
+-----+-----+
```

Captura D.5. 07 usuario backup db

```
alunne@db-santino:~$ sudo mariadb -e "SELECT User,Host FROM mysql.user WHERE User='dasc_backup';"
+-----+-----+
| User          | Host                |
+-----+-----+
| dasc_backup   | 192.168.60.30      |
+-----+-----+
```

Captura D.6. 08 install backup services ok

```
=====
Servidor de Backups + Servicios instalado
APP_USER=dasc
DB_HOST=192.168.60.20
DB_NAME=employees
Backups en: /home/dasc/backups
Scripts: /usr/local/bin/backups_api.sh y /usr/local/bin/servicios_api.sh
SSH listo para autenticación por contraseña y clave pública
=====
```

Captura D.7. 11 install api ok

```
=====
Instalación completada
Panel instalado en: /opt/dasc/api
Servicio: dasc-api
Usuario admin del panel: admin
SSH aislado de DASC: /opt/dasc/api/.ssh
Clave usada: /opt/dasc/api/.ssh/id_rsa_dasc
Known hosts usado: /opt/dasc/api/.ssh/known_hosts_dasc
SSH automático configurado contra: 192.168.60.30
URL local: http://127.0.0.1:8000
URL red: http://<IP_DEL_SERVIDOR>:8000
=====
```

Captura D.8. 12 dasc api status

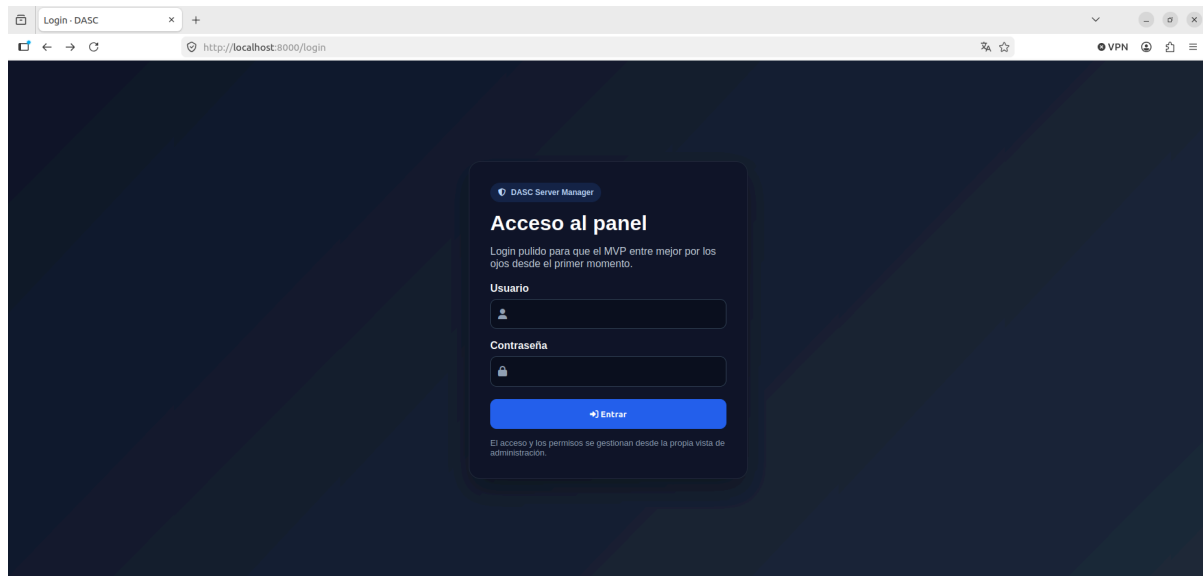
```
alunne@maquinaapi:~/dasc-server-manager/deploy/api$ systemctl status dasc-api
● dasc-api.service - DASC Panel + API (FastAPI/Uvicorn)
   Loaded: loaded (/etc/systemd/system/dasc-api.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-05-07 19:20:38 CEST; 47s ago
     Main PID: 3884 (uvicorn)
       Tasks: 1 (limit: 4597)
      Memory: 34.7M
         CPU: 628ms
    CGroup: /system.slice/dasc-api.service
            └─3884 /opt/dasc/api/venv/bin/python /opt/dasc/api/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000

may 07 19:20:38 maquinaapi systemd[1]: Started DASC Panel + API (FastAPI/Uvicorn).
may 07 19:20:39 maquinaapi uvicorn[3884]: INFO: Started server process [3884]
may 07 19:20:39 maquinaapi uvicorn[3884]: INFO: Waiting for application startup.
may 07 19:20:39 maquinaapi uvicorn[3884]: INFO: Application startup complete.
may 07 19:20:39 maquinaapi uvicorn[3884]: INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
may 07 19:20:40 maquinaapi uvicorn[3884]: Error guardando evento: (1130, "Host '192.168.60.10' is not allowed to connect to this MariaDB server")
may 07 19:20:40 maquinaapi uvicorn[3884]: INFO: 127.0.0.1:43666 - "HEAD / HTTP/1.1" 303 See Other
```

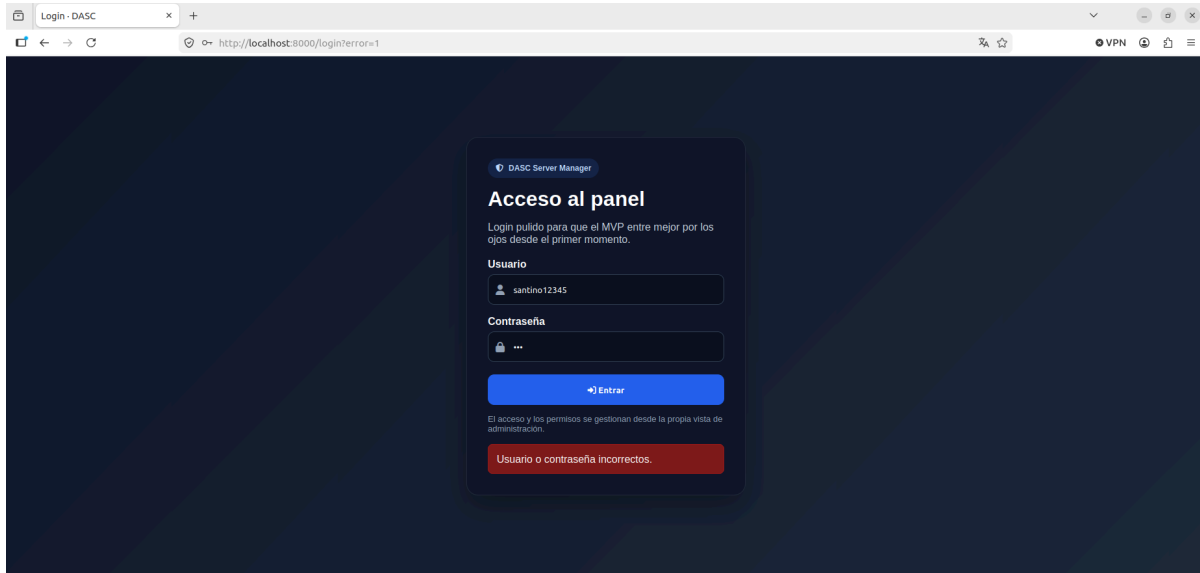
Captura D.9. 13 dependencias python

```
alumno@maquinaapi:/opt/dasc/api$ source venv/bin/activate
(venv) alumno@maquinaapi:/opt/dasc/api$ pip freeze | grep -E "fastapi|uvicorn|jinja2|pymysql|paramiko|python-dotenv"
fastapi==0.135.1
paramiko==2.9.3
python-dotenv==1.2.2
uvicorn==0.42.0
```

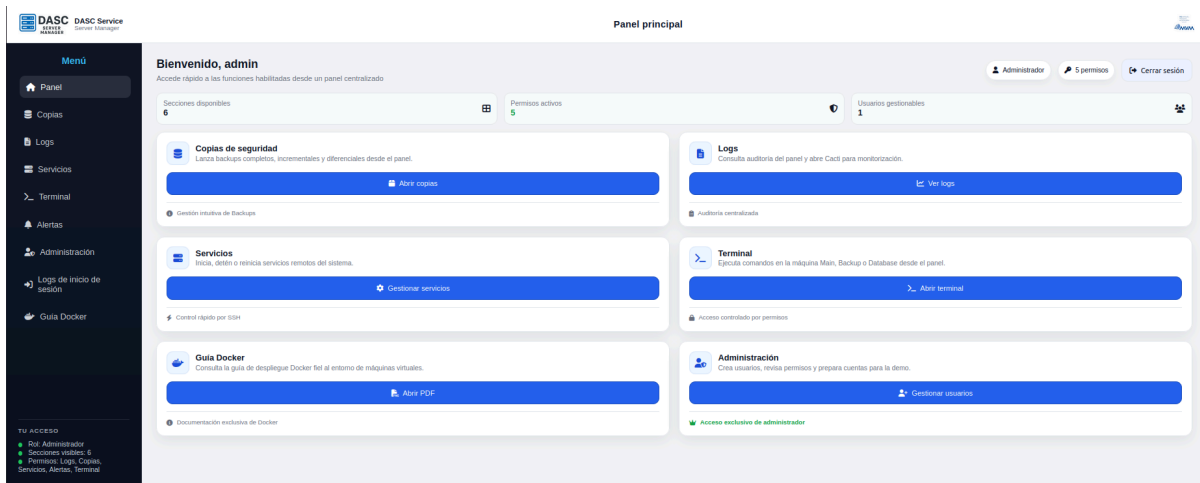
Captura D.10. 14 login



Captura D.11. 15 login error



Captura D.12. 16



Captura D.13. 17 admin usuarios

The screenshot shows the 'Administración de usuarios' interface. On the left is a dark sidebar menu with options: Panel, Copias, Logs, Servicios, Terminal, Alertas, Administración (selected), and Logs de inicio de sesión. The main area has a header with 'DASC Service Manager' and 'Administración de usuarios'. Below the header, there's a status bar showing 'Usuarios totales: 1', 'Permisos disponibles: 5', and 'Cuenta protegida: admin'. The 'Crear nuevo usuario' form is visible, with fields for 'Usuario' (containing 'Ej: tecnico1') and 'Contraseña' (containing 'Contraseña del usuario'). Below the form are checkboxes for 'Permisos del panel': Logs, Copias, Servicios, Alertas, and Terminal. A 'Crear usuario' button is at the bottom of the form. Below the form is a section titled 'Usuarios creados' with a table showing the current state of the system. The table has columns for 'USUARIO', 'PERMISOS', and 'ACCIONES'. It lists the 'admin' user with 'Todos' permissions and 'No eliminable' actions. A message at the bottom of the table states 'Todavía no hay usuarios creados.'

Menú

- Panel
- Copias
- Logs
- Servicios
- Terminal
- Alertas
- Administración**
- Logs de inicio de sesión

Administración de usuarios

Usuarios totales: 1 | Permisos disponibles: 5 | Cuenta protegida: admin

Crear nuevo usuario

Usuario: Ej: tecnico1 | Contraseña: Contraseña del usuario

Permisos del panel

☐ Logs ☐ Copias ☐ Servicios ☐ Alertas ☐ Terminal

Crear usuario

Usuarios creados
Listado actual del sistema.

USUARIO	PERMISOS	ACCIONES
admin	Todos	No eliminable

Todavía no hay usuarios creados.

Captura D.14. 18 crear usuario limitado

The screenshot shows the 'Administración de usuarios' interface after creating a new user. A green notification bar at the top says 'Usuario creado correctamente'. The status bar now shows 'Usuarios totales: 2', 'Permisos disponibles: 5', and 'Cuenta protegida: admin'. The 'Crear nuevo usuario' form is still visible. The 'Usuarios creados' table now has two rows: 'admin' with 'Todos' permissions and 'No eliminable' actions, and 'tecnico1' with 'backups, servicios' permissions and an 'Eliminar' button.

Usuario creado correctamente

Administración de usuarios

Usuarios totales: 2 | Permisos disponibles: 5 | Cuenta protegida: admin

Crear nuevo usuario

Usuario: Ej: tecnico1 | Contraseña: Contraseña del usuario

Permisos del panel

☐ Logs ☐ Copias ☐ Servicios ☐ Alertas ☐ Terminal

Crear usuario

Usuarios creados
Listado actual del sistema.

USUARIO	PERMISOS	ACCIONES
admin	Todos	No eliminable
tecnico1	backups, servicios	Eliminar

Captura D.15. 19 usuario limitado menu

Panel principal

Bienvenido, tecnico1

Accede rápido a las funciones habilitadas desde un panel centralizado

Secciones disponibles: 2

Permisos activos: 2

Copias de seguridad

Lanza backups completos, incrementales y diferenciales desde el panel.

Abrir copias

Servicios

Inicia, detén o reinicia servicios remotos del sistema.

Gestionar servicios

Control remoto por SSH

TU ACCESO

- Ref. Usuario
- Secciones visibles: 2
- Permisos: Copias, Servicios

Captura D.16. 21 admin login logs

Auditoría de login y logout

Registro privado del administrador con usuario, fecha, IP, navegador y resultado.

Registros mostrados: 7

Correctos: 6

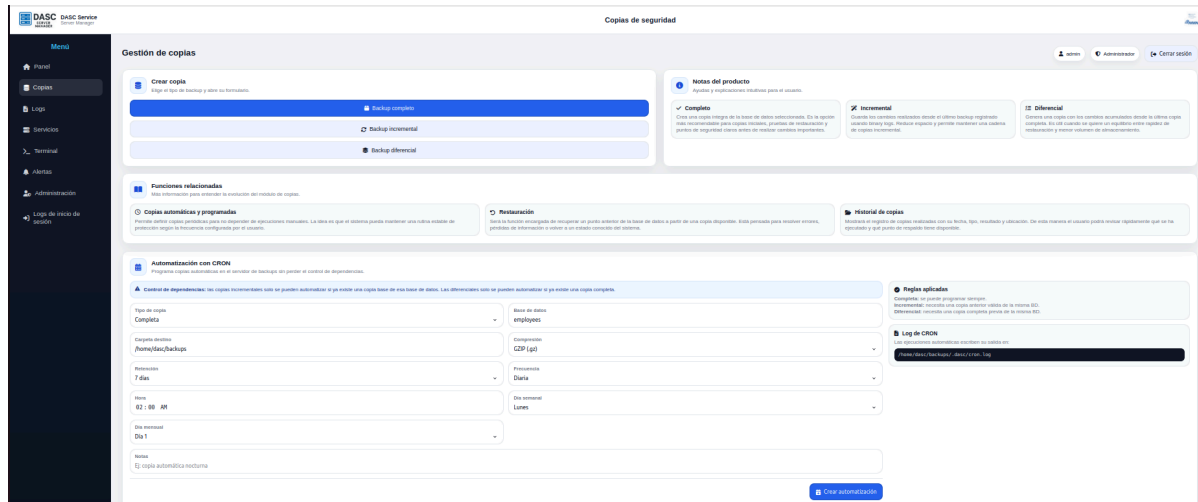
Fallidos: 1

Movimientos de autenticación

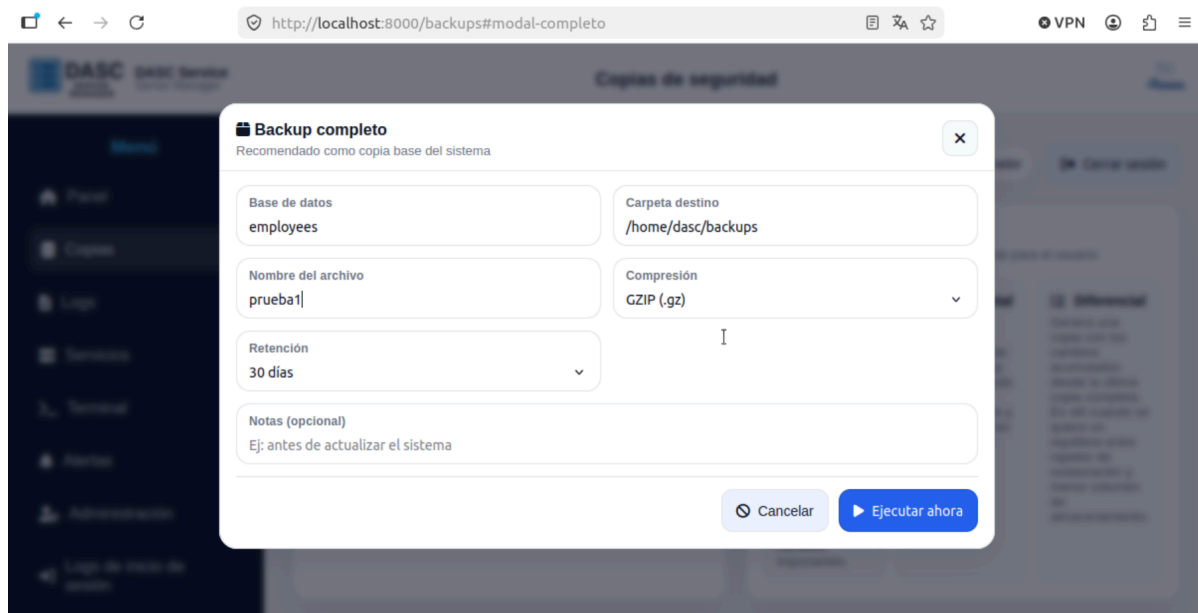
Se guardan los inicios de sesión correctos, intentos fallidos y cierres de sesión de cualquier usuario.

FECHA	ACCIÓN	RESULTADO	USUARIO	ROL	IP ORIGEN	NAVEGADOR / CLIENTE	DETALLE
2026-05-15 17:48:00	LOGIN	OK	admin	Administrador	127.0.0.1	Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0	Inicio de sesión correcto
2026-05-15 17:47:51	LOGOUT	OK	tecnico1	Usuario	127.0.0.1	Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0	Cierre de sesión correcto
2026-05-15 17:45:51	LOGIN	OK	tecnico1	Usuario	127.0.0.1	Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0	Inicio de sesión correcto
2026-05-15 17:45:37	LOGIN	ERROR	tecnico1	No autenticado	127.0.0.1	Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0	Intento de inicio de sesión fallido
2026-05-15 17:45:27	LOGOUT	OK	admin	Administrador	127.0.0.1	Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:150.0) Gecko/20100101 Firefox/150.0	Cierre de sesión correcto
2026-05-15 07:46:33	LOGIN	OK	admin	Administrador	10.2.0.2	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36	Inicio de sesión correcto
2026-05-15 07:46:02	LOGIN	OK	admin	Administrador	10.2.0.2	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 Safari/537.36	Inicio de sesión correcto

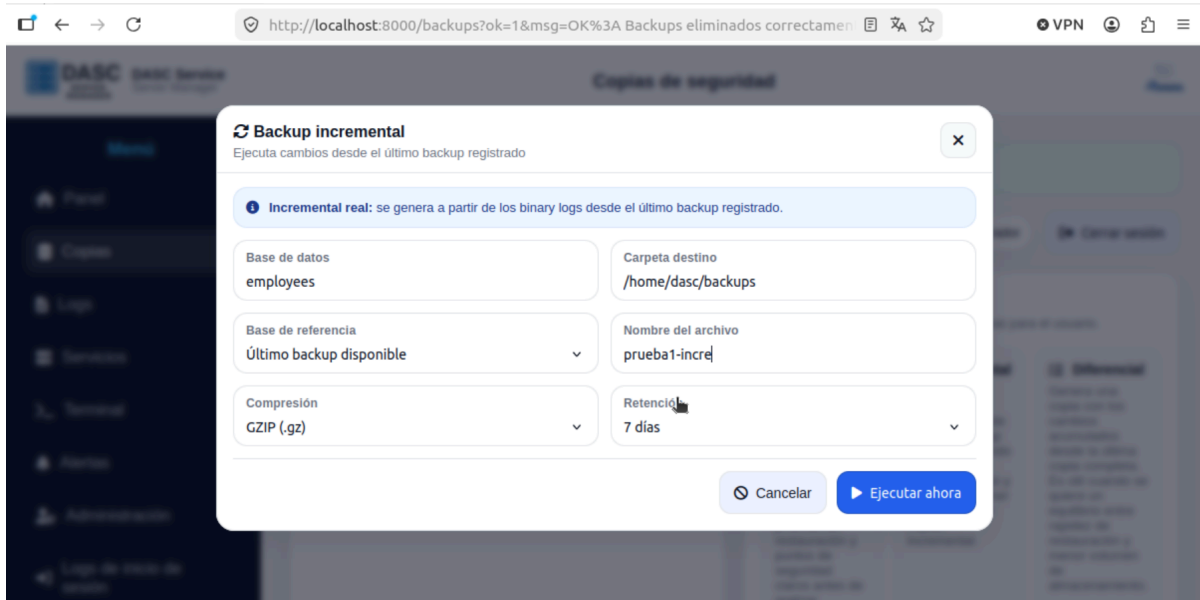
Captura D.17. 23



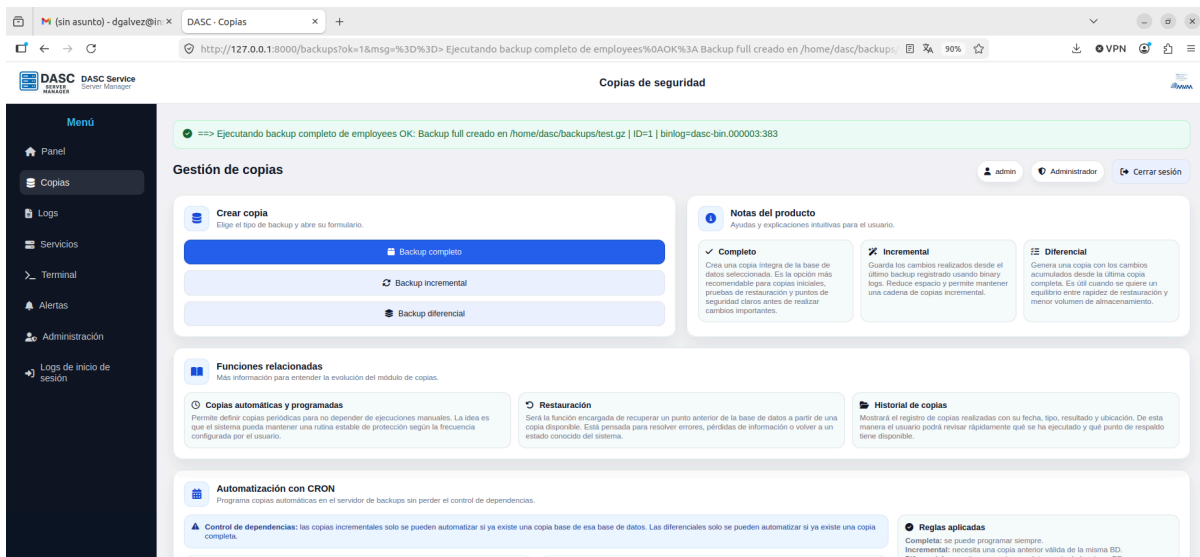
Captura D.18. 23 backup full ok



Captura D.19. 24 backup incremental ok



Captura D.20. 24 captura24



Captura D.21. 25 captura25 incremental

The screenshot shows the DASC Service web interface. At the top, a green status bar indicates a successful incremental backup: "Ejecutando backup incremental de employees => Desde dasc-bin.000003:383 hasta dasc-bin.000003:383 OK: Backup incremental creado en /home/dasc/backups/test.gz | ID=2 | base=1 | rango=dasc-bin.000003:383-dasc-bin.000003:383".

The main section is titled "Gestión de copias" (Copy Management) and includes a sidebar menu with options like Panel, Copias, Logs, Servicios, Terminal, Alertas, Administración, and Logs de inicio de sesión.

Under "Gestión de copias", there are three main buttons: "Backup completo", "Backup incremental", and "Backup diferencial". The "Backup incremental" button is highlighted.

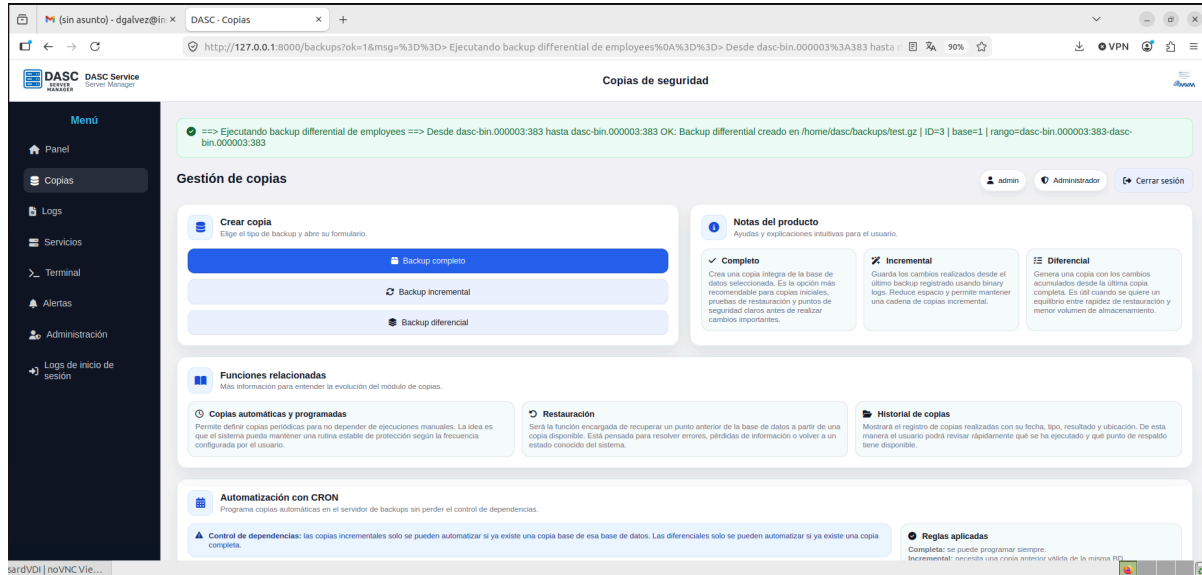
Below these buttons, there are several informational sections:

- Notas del producto** (Product Notes): Explains the different backup types: Completo (Complete), Incremental (Incremental), and Diferencial (Differential).
- Funciones relacionadas** (Related Functions): Includes "Copias automáticas y programadas" (Automatic and scheduled copies), "Restauración" (Restoration), and "Historial de copias" (Copy history).
- Automatización con CRON** (Automation with CRON): Explains how to schedule backups using CRON.
- Reglas aplicadas** (Applied rules): A section for backup rules.

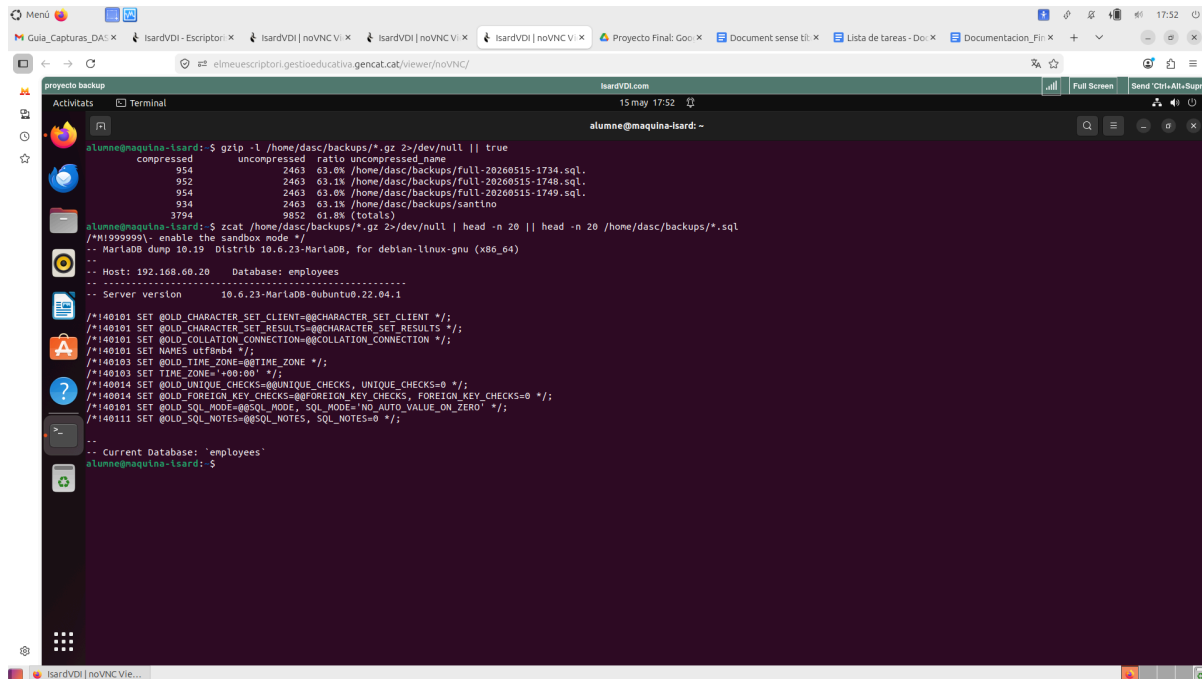
Captura D.22. 26 archivos backups remoto

```
alumno@maquinabackups:~$ ls -lh /home/dasc/backups/
total 16K
drwxrwxr-x 2 dasc dasc 4,0K may 15 06:21 pre-restore
-rw-rw-r-- 1 dasc dasc 318 may 15 18:08 prueba1-diff.gz
-rw-rw-r-- 1 dasc dasc 1010 may 15 17:53 prueba1.gz
-rw-rw-r-- 1 dasc dasc 319 may 15 18:07 prueba1-incre.gz
```

Captura D.23. 26 diferencial



Captura D.24. 27 captura27



Captura D.27. 28 descarga backup panel

http://localhost:8000/backups?ok=1&msg=%3D%3D> Ejecutando backup differ

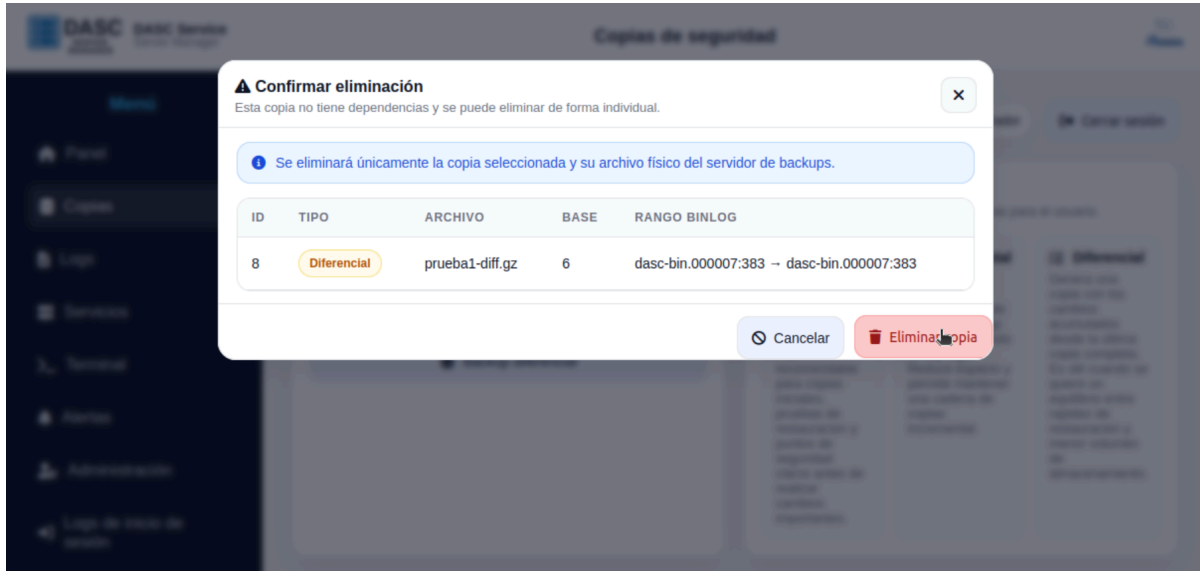
prueba1.gz

Completed — 1.0 KB

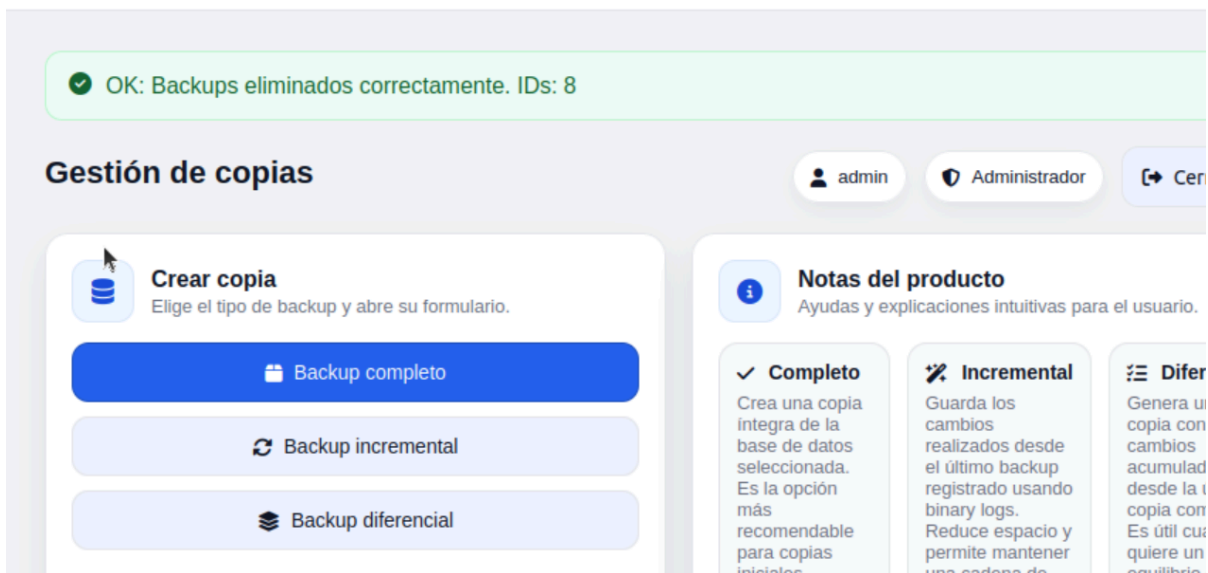
Show all downloads

ID	FECHA	TIP					ACCIONES
8	2026-05-15 18:08:19	Diferencial	employees	prueba1-diff.gz	6	dasc-bin.000007:383 → dasc-bin.000007:383	Descargar Restaurar Eliminar
7	2026-05-15 18:07:48	Incremental	employees	prueba1-incre.gz	6	dasc-bin.000007:383 → dasc-bin.000007:383	Descargar Restaurar Eliminar
6	2026-05-15 17:53:40	Completo	employees	prueba1.gz	-	-	Descargar Restaurar Eliminar

Captura D.28. 29 eliminacion backup confirmacion



Captura D.29. 29 eliminacion backup confirmacion2



Captura D.30. 30

DASC Service
Backup & Recovery Manager

Copias de seguridad

Panel

Copias

Logs

Servicios

Terminal

Alertas

Administración

Logs de inicio de sesión

==> Ejecutando backup completo de employees OK: Backup full creado en /home/dasc/backups/prueba1.gz | ID=1 | binlog=dasc-bin.000003:383

Gestión de copias

Crear copia

Elige el tipo de backup y abre su formulario.

- Backup completo
- Backup incremental
- Backup diferencial

Notas del producto

Ayudas y explicaciones intuitivas para el usuario.

✓ Completo

Crea una copia íntegra de la base de datos seleccionada. Es la opción más recomendable para copias iniciales, pruebas de restauración y puntos de seguridad claros antes de realizar cambios importantes.

⚡ Incremental

Guarda los cambios realizados desde el último backup registrado usando binary logs. Reduce espacio y permite mantener una cadena de copias incrementales.

🔄 Diferencial

Genera una copia con los cambios acumulados desde la última copia completa. Es útil cuando se quiere un equilibrio entre rapidez de restauración y menor volumen de almacenamiento.

Funciones relacionadas

Más información para entender la evolución del módulo de copias.

🕒 Copias automáticas y programadas

Permite definir copias periódicas para no depender de ejecuciones manuales. La idea es que el sistema pueda mantener una rutina estable de protección según la frecuencia configurada por el usuario.

🔄 Restauración

Simla la función encargada de recuperar un punto anterior de la base de datos a partir de una copia disponible. Está pensada para resolver errores, pérdidas de información o volver a un estado conocido del sistema.

📅 Historial de copias

Mostrará el registro de copias realizadas con su fecha, tipo, resultado y ubicación. De esta manera el usuario podrá revisar rápidamente qué se ha replicado y qué punto de respaldo tiene disponible.

Automatización con CRON

Programa copias automáticas en el servidor de backups sin perder el control de dependencias.

▲ Control de dependencias: las copias incrementales solo se pueden automatizar si ya existe una copia base de esa base de datos. Los diferenciales solo se pueden automatizar si ya existe una copia completa.

Tipo de copia	Base de datos
Completa	employees

Reglas aplicadas

Completas: se puede programar siempre.
Incremental: necesita una copia anterior válida de la misma BD.
Diferencial: necesita una copia completa previa de la misma BD.

Captura D.31. 30 restore

DASC

DASC Service

Server Manager

Menú

Panel

Copias

Logs

Servicios

Terminal

Alertas

Administración

Logs de inicio de sesión

Copias de seguridad

admin

Administrador

Cerrar sesión

Crear copia

Elige el tipo de backup y abre su formulario.

Backup completo

Backup incremental

Backup diferencial

Notas del producto

Ayudas y explicaciones intuitivas para el usuario.

✓ Completo

Crea una copia íntegra de la base de datos seleccionada. Es la opción más recomendable para copias iniciales, pruebas de restauración y puntos de seguridad claros antes de realizar cambios importantes.

✗ Incremental

Guarda los cambios realizados desde el último backup registrado usando binary logs. Reduce espacio y permite mantener una cadena de copias incremental.

⚡ Diferencial

Genera una copia con los cambios acumulados desde la última copia completa. Es útil cuando se quiere un equilibrio entre rapidez de restauración y menor volumen de almacenamiento.

Funciones relacionadas

Más información para entender la evolución del módulo de copias.

🕒 Copias automáticas y programadas

Permite definir copias periódicas para no depender de ejecuciones manuales. La idea es que el sistema pueda mantener una rutina estable de protección según la frecuencia configurada por el usuario.

🔄 Restauración

Será la función encargada de recuperar un punto anterior de la base de datos a partir de una copia disponible. Está pensada para resolver errores, pérdidas de información o volver a un estado conocido del sistema.

📜 Historial de copias

Mostrará el registro de copias realizadas con su fecha, tipo, resultado y ubicación. De esta manera el usuario podrá revisar rápidamente qué se ha ejecutado y qué punto de respaldo tiene disponible.

Automatización con CRON

Programa copias automáticas en el servidor de backups sin perder el control de dependencias.

⚠️ Control de dependencias:

las copias incrementales solo se pueden automatizar si ya existe una copia base de esa base de datos. Las diferenciales solo se pueden automatizar si ya existe una copia completa.

Reglas aplicadas

Completo: se puede programar siempre.
Incremental: necesita una copia anterior válida de la misma BD.
Diferencial: necesita una copia completa previa de la misma BD.

Tipo de copia

Completa

Base de datos

employees

Captura D.32. 30 restore preview confirm

 **Confirmar restauración**

La base de datos será restaurada al punto seleccionado.



 **Atención:** esta operación modificará la base de datos **employees**. Antes de restaurar, el sistema creará una copia preventiva automática del estado actual.

 Esta restauración solo necesita aplicar la copia completa seleccionada.

ORDEN	ID	TIPO	ARCHIVO	BASE	RANGO BINLOG
1	6	Completo	prueba1.gz	-	-

 Cancelar

 Restaurar backup

Captura D.33. 31 automatizacion backups

 **Automatización con CRON**

Programa copias automáticas en el servidor de backups sin perder el control de dependencias.

 **Control de dependencias:** las copias incrementales solo se pueden automatizar si ya existe una copia base de esa base de datos. Las diferenciales solo se pueden automatizar si ya existe una copia completa.

Tipo de copia
Completa

Base de datos
employees

Carpeta destino
/home/dasc/backups

Compresión
GZIP (.gz)

Retención
7 días

Frecuencia
Diaria

Hora
02 : 00 AM

Día semanal
Lunes

Día mensual
Día 1

Notas
Ej: copia automática nocturna

 Crear automatización

 **Reglas aplicadas**

Completa: se puede programar siempre.
Incremental: necesita una copia anterior válida de la misma BD.
Diferencial: necesita una copia completa previa de la misma BD.

 **Log de CRON**

Las ejecuciones automáticas escriben su salida en:

/home/dasc/backups/.dasc/cron.log

 **Automatizaciones activas**

ID	TIPO	BD	FRECUENCIA	DESTINO	CREADA POR	ACCIONES
auto-20260515182538	Completa	employees	Cada día a las 02:00 0 2 * * *	/home/dasc/backups	admin	 Eliminar

Captura D.34. 33

DASC Service
Server Manager

Menú

Panel

Copias

Logs

Servicios

Terminal

Alertas

Administración

Logs de inicio de sesión

RESUMEN

Activos: 3

No activos: 1

Total listados: 4

Logs y monitorización

admin

Cerrar sesión

Auditoría del sistema

Eventos cargados: 40

Resultado OK: 32

Resultado ERROR: 8

Monitorización externa

Acceso directo a Cacti desde el panel.

Apert Cacti

Eventos de DASC Web

Últimos 100 registros de auditoría almacenados en base de datos.

ID	FECHA	ORIGEN	TIPO	USUARIO	IP	RECURSO	RESULTADO	DETALLE
40	2026-05-15 16:24:40	dasc-web	acceso	admin	192.168.60.1	GET /backups	OK	HTTP 200
39	2026-05-15 16:24:39	dasc-web	backup	admin	192.168.60.1	POST /backups/restore/confirm	OK	HTTP 303
38	2026-05-15 16:24:32	dasc-web	backup	admin	192.168.60.1	POST /backups/restore/preview	OK	HTTP 200
36	2026-05-15 16:23:55	dasc-web	backup	admin	192.168.60.1	POST /backups/run	OK	HTTP 303
37	2026-05-15 16:23:55	dasc-web	acceso	admin	192.168.60.1	GET /backups	OK	HTTP 200
33	2026-05-15 16:23:32	dasc-web	acceso	admin	192.168.60.1	GET /backups	OK	HTTP 200
34	2026-05-15 16:23:32	dasc-web	acceso	admin	192.168.60.1	GET /static/css/backups.css	OK	HTTP 200

Captura D.35. 36

DASC Service
Server Manager

Menú

Panel

Copias

Logs

Servicios

Terminal

Alertas

Administración

Logs de inicio de sesión

RESUMEN

Activos: 3

No activos: 1

Total listados: 4

Gestión de servicios

admin

Cerrar sesión

Servicios del sistema

Servicios: 4

Activos: 3

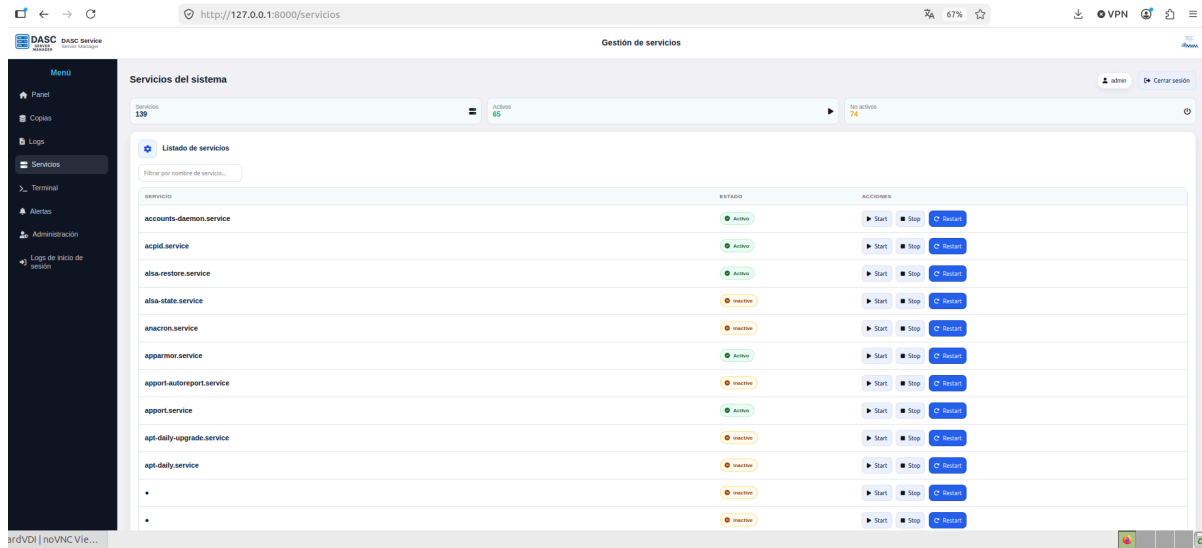
No activos: 1

Listado de servicios

Filtrar por nombre de servicio...

SERVICIO	ESTADO	ACCIONES
ssh.service	Activo	Start Stop Restart
cron.service	Activo	Start Stop Restart
backup-worker.service	Activo	Start Stop Restart
cacti.service	Inactivo	Start Stop Restart

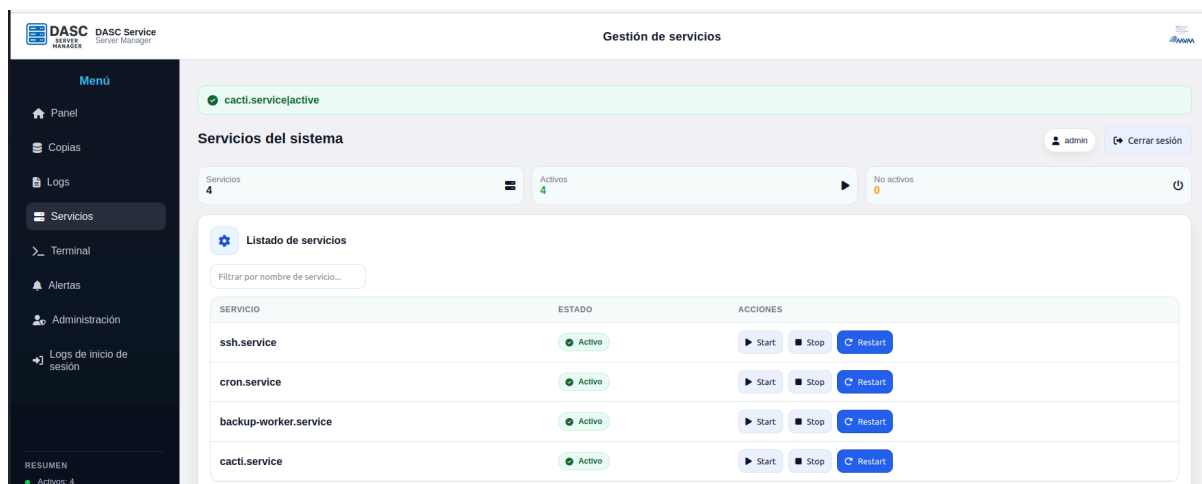
Captura D.36. 36 captura36



The screenshot shows the DASC Service Manager web interface. The left sidebar contains a menu with options: Panel, Copias, Logs, Servicios (selected), Terminal, Alertas, Administración, and Logs de inicio de sesión. The main content area is titled 'Gestión de servicios' and 'Servicios del sistema'. It displays a table of system services with columns for 'servicio', 'estado', and 'acciones'. The table lists 13 services, all of which are 'Activo' (Active). The actions for each service include Start, Stop, and Restart buttons. The top right of the interface shows the user 'admin' and a 'Cerrar sesión' (Logout) button.

SERVICIO	ESTADO	ACCIONES
accounts-daemon.service	Activo	Start Stop Restart
acpid.service	Activo	Start Stop Restart
alsa-restore.service	Activo	Start Stop Restart
alsa-state.service	Inactivo	Start Stop Restart
anacron.service	Inactivo	Start Stop Restart
apparmor.service	Activo	Start Stop Restart
apport-autoreport.service	Inactivo	Start Stop Restart
apport.service	Activo	Start Stop Restart
apt-daily-upgrade.service	Inactivo	Start Stop Restart
apt-daily.service	Inactivo	Start Stop Restart
*	Inactivo	Start Stop Restart
*	Inactivo	Start Stop Restart

Captura D.37. 37



The screenshot shows the DASC Service Manager web interface. The left sidebar contains a menu with options: Panel, Copias, Logs, Servicios (selected), Terminal, Alertas, Administración, and Logs de inicio de sesión. The main content area is titled 'Gestión de servicios' and 'Servicios del sistema'. It displays a table of system services with columns for 'servicio', 'estado', and 'acciones'. The table lists 4 services, all of which are 'Activo' (Active). The actions for each service include Start, Stop, and Restart buttons. The top right of the interface shows the user 'admin' and a 'Cerrar sesión' (Logout) button.

SERVICIO	ESTADO	ACCIONES
ssh.service	Activo	Start Stop Restart
cron.service	Activo	Start Stop Restart
backup-worker.service	Activo	Start Stop Restart
cacti.service	Activo	Start Stop Restart

Captura D.38. 38

```
lsard@ubuntu: ~/Baixades/dasc-server-manager-docker/dasc-server-manager-docker-f1e1$ sudo systemctl status cron --no-pager
● cron.service - Regular background program processing daemon
   Loaded: loaded (/lib/systemd/system/cron.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-05-15 18:01:14 CEST; 15min ago
     Docs: man:cron(8)
    Main PID: 612 (cron)
      Tasks: 1 (limit: 19093)
    Memory: 424.0K
       CPU: 8ms
    CGroup: /system.slice/cron.service
            └─612 /usr/sbin/cron -f -P

de maig 15 18:01:14 ubuntu systemd[1]: Started Regular background program p_mon.
de maig 15 18:01:14 ubuntu cron[612]: (CRON) INFO (pidfile fd = 3)
```

Captura D.39. 38 captura38

```

alumn@magnum-lsard: /dasc-server-manager$ systemctl status cron --no-pager
● cron.service - Regular background program processing daemon
   Loaded: loaded (/lib/systemd/system/cron.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2026-05-15 17:24:20 CEST; 43min ago
     Docs: man:cron(8)
    Main PID: 595 (cron)
      Tasks: 1 (limit: 4596)
     Memory: 464.0K
        CPU: 19ms
    CGroup: /system.slice/cron.service
            └─595 /usr/sbin/cron -f -P

may 15 17:24:20 magnum-lsard systemd[1]: Started Regular background program processing daemon.
may 15 17:24:20 magnum-lsard cron[595]: (CRON) INFO (pidfile fd = 3)
may 15 17:24:20 magnum-lsard cron[595]: (CRON) INFO (Running @reboot jobs)
may 15 17:30:01 magnum-lsard CRON[2164]: pam_unix(cron:session): session opened for user root(uid=0) by (uid=0)
may 15 17:30:01 magnum-lsard CRON[2165]: (root) CMD [ -x /etc/init.d/anacron ] && if [ ! -d /run/systemd/system ]; then /usr/sbin/lnvke-rc.d anacron start >/dev/null; fi)
may 15 17:30:01 magnum-lsard CRON[2164]: pam_unix(cron:session): session closed for user root
alumn@magnum-lsard: /dasc-server-manager$ systemctl is-active cron
active
alumn@magnum-lsard: /dasc-server-manager$

```

Captura D.40. 39

Gestión de Alertas

- Panel
- Carpas
- Logs
- Servicios
- Terminal
- Alertas**
- Administración
- Log de inicio de sesión

Alertas y destinatarios

Gestiona Telegram, ingresa grupos, usuarios y entornos del sistema.

Usuarios activos

1

Usuarios registrados en IQOS

Entornos configurados

0

Entorno añadido correctamente

Entorno con error

1

Error: ID incorrecto, chat_id no encontrado

Canal Telegram

Estado general del canal y posibles errores.

Nombre	Estado	Última Notificación
Telegram	Activo	2024-05-15 16:14:52

Desactivar canal [Enviar prueba al destino por defecto](#)

Detectar chats recientes

Haz clic en el bot o añade el bot a un grupo y luego pulsa detectar.

[Añadir bot en Telegram](#) [Detectar chats recientes](#) [Abrir guía PDF](#)

USUARIO / NOMBRE	CHAT ID	TIPO	ACTION
Todavía no hay chats detectados. Haz clic en el bot o pulsa detectar.			

Destinatarios Telegram

Añade usuarios o grupos manualmente, o déjalo que detecte automáticamente.

Recomendado para el API: crear un grupo de Telegram por empresa, añadir el bot y usar ese único chat_id. También puedes añadir usuarios individuales si cada persona ha hablado con el bot antes.

Empresa:

Nombre visible:

Tipo:

Chat ID:

+ Añadir destinatario
+ Manual chat_id (PDF)

EMPRESA	NOMBRE	TIPO	USUARIO	DESTINO	ESTADO	DEFAULT	ACTIONS
No hay destinatarios guardados todavía.							

Captura D.41. 39 alertas telegram(1)

DASC

Server Manager

Gestión de Alertas

Menú

Panel

Copias

Logs

Servicios

Alertas

Administración

Centro de Alertas

Configura Telegram, revisa reglas y consulta el historial de envíos

Usuarios: admin

Cerrar sesión

Eventos totales

10

Eventos internos generados por el panel

Entregas realizadas

10

Intentos de envío a canales configurados

Última entrega

2026-05-15 15:53:52

Último registro guardado en historial

Canales disponibles

Estado actual de Telegram y resto de canales configurables

CANAL	DESTINO	ESTADO	ACCIÓN
Telegram	5883797612	Activo	Desactivar

Enviar prueba Telegram

Estado del módulo

Resumen rápido para la demo del MVP

CANAL INICIAL

Telegram

MOTOR INTERNO

Eventos + reglas + entregas

BASE LOCAL

SQLite

ÁMBITOS

Backups, servicios y API

Consejo para la presentación:

enseña primero esta pantalla, luego pulsa "Enviar prueba", después fuerza un backup o una acción de servicio y vuelve aquí para mostrar el historial de entregas.

Captura D.42. 39 alertas telegram

Reglas de alertas

Activa o desactiva qué eventos generan notificación

EVENTO	SEVERIDAD	CANAL	ESTADO	ACCIÓN
api.error	Critical	telegram	Activa	Desactivar
backup.error	Critical	telegram	Activa	Desactivar
backup.ok	Info	telegram	Activa	Desactivar
service.error	Critical	telegram	Activa	Desactivar
service.ok	Info	telegram	Activa	Desactivar

Historial de entregas

Últimas alertas enviadas o fallidas

FECHA	EVENTO	TÍTULO	CANAL	ESTADO
2026-05-15 15:53:52	backup.ok	Backup completado	telegram	OK
2026-05-15 15:49:28	backup.ok	Backup completado	telegram	OK
2026-05-15 15:48:21	backup.ok	Backup completado	telegram	OK

Captura D.43. 40 alertas detectar destinatario

**Canal Telegram**
Estado general del canal y prueba manual.

CANAL
Telegram

ESTADO
Activo

DESTINO POR DEFECTO
Sin destino todavía

ÚLTIMA ENTREGA
2026-05-15 16:25:38

Desactivar canal

Enviar prueba al destino por defecto

**Detectar chats recientes**
Haz /start al bot o añade el bot a un grupo y luego pulsa detectar.

Abrir bot en Telegram

Detectar chats recientes

Abrir guía PDF

USERNAME / NOMBRE	CHAT ID	TIPO	ACCIÓN
@XLHZMYqrQm	5883797612	 Usuario	+ Añadir

Captura D.44. 41 alertadeprueba

**Canales disponibles**
Estado actual de Telegram y resto de canales configurables

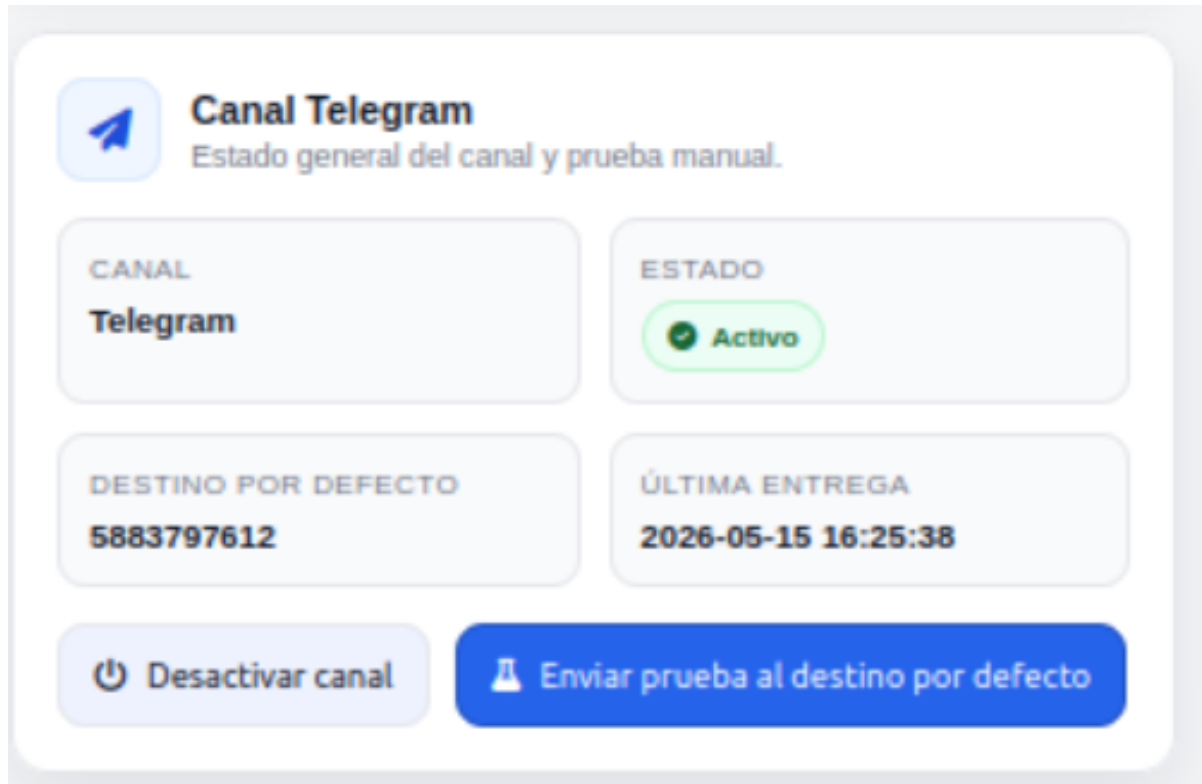
CANAL	DESTINO	ESTADO	ACCIÓN
Telegram telegram	5883797612	 Activo	Desactivar

Enviar prueba Telegram

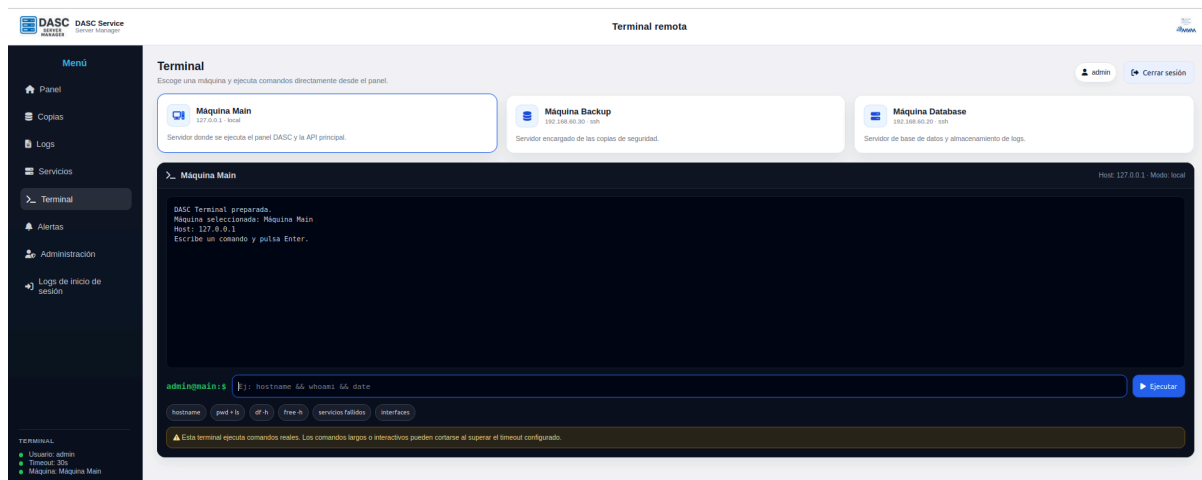
Captura D.45. 41 alertas



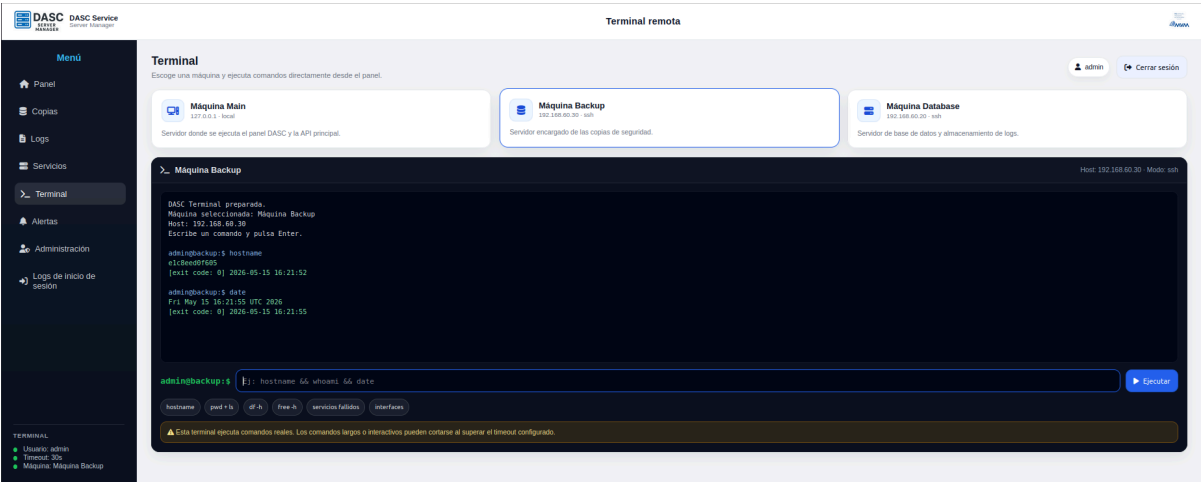
Captura D.46. 41 telegram alerta prueba1



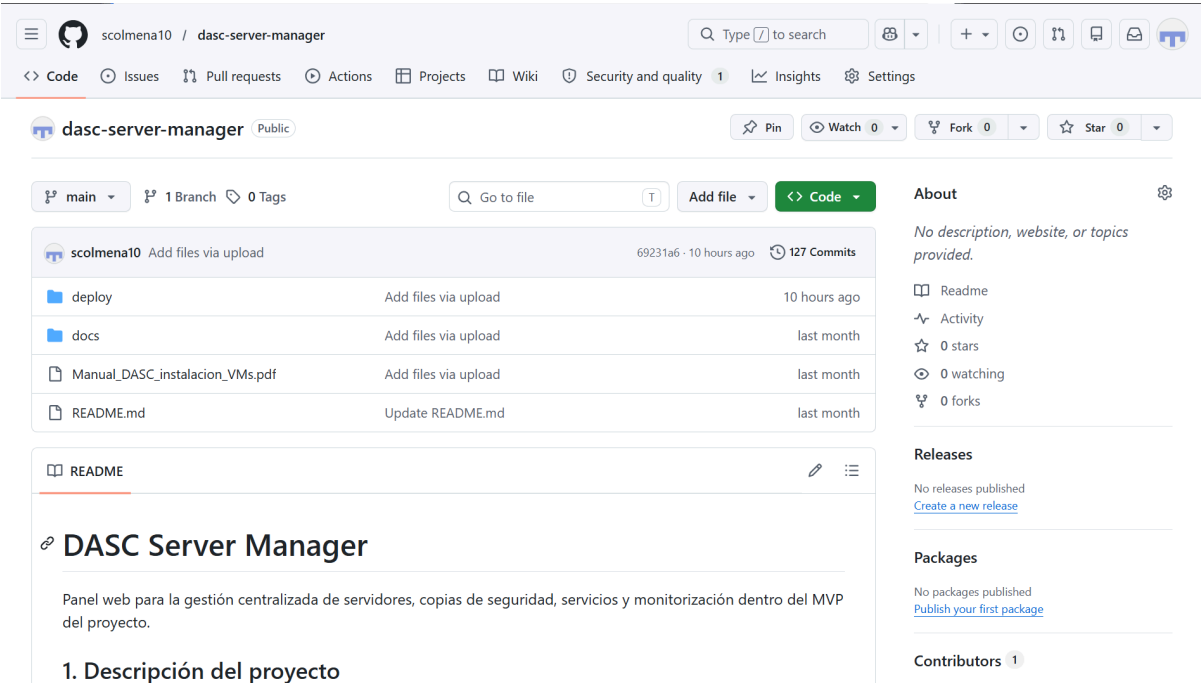
Captura D.47. 43



Captura D.48. 44



Captura D.49. 45 github estructura



Captura D.50. 46

```
tsard@ubuntu:~/Baixades/dasc-server-manager-docker/dasc-server-manager-docker-fiel$ sudo docker compose build
[+] Building 219.9s (56/56) FINISHED                                docker:default
=> [cacti internal] load build definition from Dockerfile            0.1s
=> => transferring dockerfile: 839B                                  0.0s
=> [mysql-db internal] load build definition from Dockerfile        0.1s
=> => transferring dockerfile: 1.0kB                                  0.0s
=> [backup-services internal] load metadata for docker.io/library/debian 1.7s
=> [mysql-db internal] load metadata for docker.io/library/mariadb:10.6 1.5s
=> [cacti internal] load .dockerignore                              0.0s
=> => transferring context: 2B                                         0.0s
=> [backup-services 1/6] FROM docker.io/library/debian:12@sha256:85019db29298555fd1a5f4b 5.5s
=> => resolve docker.io/library/debian:12@sha256:85019db29298555fd1a5f4b 0.0s
=> => sha256:9104793a826a01456b0ba567a92064e85a0298166 48.49MB / 48.49MB 1.3s
=> => sha256:85019db29298555fd1a5f4b57673ae989414a98841 8.52kB / 8.52kB 0.0s
=> => sha256:8c1818d2706019b2ee6237441b3fb8691b92b5705d 1.02kB / 1.02kB 0.0s
=> => sha256:49c51d813529e19b096cbf96f27f4c8f56e33ac4e2e4e94 453B / 453B 0.0s
=> => extracting sha256:9104793a826a01456b0ba567a92064e85a029816661fbb45 3.9s
=> [cacti internal] load build context                              0.0s
=> => transferring context: 4.22kB                                     0.0s
=> [mysql-db internal] load .dockerignore                          0.1s
=> => transferring context: 2B                                         0.0s
=> [mysql-db 1/9] FROM docker.io/library/mariadb:10.6@sha256:16092512d3 10.1s
=> => resolve docker.io/library/mariadb:10.6@sha256:16092512d3b105aca865 0.0s
=> => sha256:734ea0539b6cd0e768583c3c1d5b193e8cd0212efb2 9.14kB / 9.14kB 0.0s
=> => sha256:734ea0539b6cd0e768583c3c1d5b193e8cd0212efb2 29.74MB / 29.74MB 2.0s
=> => sha256:16092512d3b105aca86586a12c704a89e8f33e369cf 5.13kB / 5.13kB 0.0s
=> => sha256:a1de2f70db77d1d366b6003d6ec4f7c5b20f09c7f51 2.48kB / 2.48kB 0.0s
=> => sha256:3eb33c94eb9cd3122b91878bb39d0b4b14de8070c2a 1.72kB / 1.72kB 0.4s
=> => sha256:956756ea33e7b12cf1569d2c7d79b4c9314715752cf 5.61MB / 5.61MB 1.0s
=> => sha256:5b0f49441d71c4a86b5004e40c593b5ba2d248ad7db597 116B / 116B 1.3s
=> => sha256:615106584724d1abe7bde94ca7682b1f92102e584a357a6 333B / 333B 1.4s
=> => sha256:72d4a84ca8f542861ffd9c9f592d9fc4684cd8d92 68.07MB / 68.07MB 3.0s
=> => sha256:4ca18155be93d8c53a41c983c46f97d51d19aafcdda 4.02kB / 4.02kB 1.7s
=> => sha256:768d5d924892f9098ffd64769012a0108eb80fbccd1 8.22kB / 8.22kB 3.0s
=> => extracting sha256:f63eb04151bcac21ad049f8d781b97b219aba392c5457907 2.3s
=> => extracting sha256:3eb33c94eb9cd3122b91878bb39d0b4b14de8070c2a73969 0.0s
=> => extracting sha256:956756ea33e7b12cf1569d2c7d79b4c9314715752cf2034d 1.2s
=> => extracting sha256:5b0f49441d71c4a86b5004e40c593b5ba2d248ad7db597d 0.0s
=> => extracting sha256:615106584724d1abe7bde94ca7682b1f92102e584a357a6e 0.0s
=> => extracting sha256:72d4a84ca8f542861ffd9c9f592d9fc4684cd8d929ee425b 3.1s
=> => extracting sha256:4ca18155be93d8c53a41c983c46f97d51d19aafcddacdddea 0.0s
=> => extracting sha256:768d5d924892f9098ffd64769012a0108eb80fbccd1b835b 0.0s
=> [mysql-db internal] load build context                          0.1s
```

Captura D.51. 47

```
tsard@ubuntu:~/Baixades/dasc-server-manager-docker/dasc-server-manager-docker-fiel$ sudo docker compose up -d
[+] Running 9/9
✔ cacti-db 8 layers [[:]] 0B/0B Pulled 2.2s
✔ f63eb04151bc Already exists 0.0s
✔ 3eb33c94eb9c Already exists 0.0s
✔ 956756ea33e7 Already exists 0.0s
✔ 5b0f49441d71 Already exists 0.0s
✔ 615106584724 Already exists 0.0s
✔ 72d4a84ca8f5 Already exists 0.0s
✔ 4ca18155be93 Already exists 0.0s
✔ 768d5d924892 Already exists 0.0s
[+] Running 5/11
:: Network dasc-server-manager-docker-fiel_dasc_lab_net Created 2.1s
:: Network dasc-server-manager-docker-fiel_cacti_net Created 2.0s
:: Volume "dasc-server-manager-docker-fiel_web_data" Created 1.9s
:: Volume "dasc-server-manager-docker-fiel_db_data" Created 1.9s
:: Volume "dasc-server-manager-docker-fiel_backups_data" Created 1.9s
:: Volume "dasc-server-manager-docker-fiel_cacti_db_data" Created 1.9s
✔ Container cacti-db Started 0.9s
✔ Container mysql-db Started 0.8s
✔ Container backup-services Started 1.2s
✔ Container cacti Started 1.3s
```

Captura D.52. 48

DASC Service

Panel

Copias

Logs

Servicios

Terminal

Alertas

Administración

Logs de inicio de sesión

Menú

Logs y monitorización

admin

Cerrar sesión

Eventos cargados

40

Resultado OK

32

Resultado ERROR

8

Monitorización externa

Acceso directo a Cacti desde el panel.

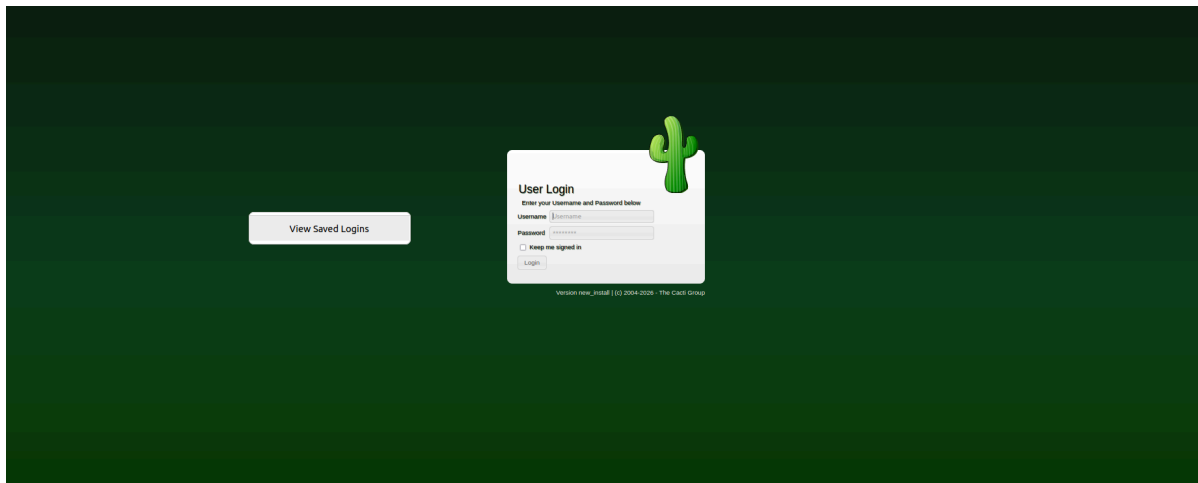
Ver Cacti

Eventos de DASC Web

Últimos 100 registros de auditoría almacenados en base de datos.

ID	FECHA	ORIGEN	TIPO	USUARIO	IP	RECURSO	RESULTADO	DETALLE
40	2026-05-15 16:24:40	dasc-web	acceso	admin	192.168.60.1	GET /backups	OK	HTTP 200
39	2026-05-15 16:24:39	dasc-web	backup	admin	192.168.60.1	POST /backups/restore/confirm	OK	HTTP 303
38	2026-05-15 16:24:32	dasc-web	backup	admin	192.168.60.1	POST /backups/restore/preview	OK	HTTP 200
36	2026-05-15 16:23:55	dasc-web	backup	admin	192.168.60.1	POST /backups/run	OK	HTTP 303
37	2026-05-15 16:23:55	dasc-web	acceso	admin	192.168.60.1	GET /backups	OK	HTTP 200
33	2026-05-15 16:23:32	dasc-web	acceso	admin	192.168.60.1	GET /backups	OK	HTTP 200
34	2026-05-15 16:23:32	dasc-web	acceso	admin	192.168.60.1	GET /staticcss/backups.css	OK	HTTP 200

Captura D.53. 49



Captura D.54. 50.1

```
lsard@ubuntu:~/Baixades/dasc-server-manager-docker/dasc-server-manager-docker-fiel$ sudo docker exec mysql-db mariadb -uroot -prootpass -e "SHOW MASTER STATUS;"
```

Captura D.55. 50

```
tsard@ubuntu: ~/tsardes/dasc-server-manager-docker/dasc-server-manager-docker-files$ sudo docker exec mysql-db mariadb -uroot -prootpass -e 'SHOW DATABASES;'
[tsard] contraseña per a tsard:
Database
dasc_logs
employees
information_schema
mysql
performance_schema
sys
tsard@ubuntu: ~/tsardes/dasc-server-manager-docker/dasc-server-manager-docker-files$
```

Se conservan aquí los diagramas alternativos o duplicados para consulta del equipo durante la preparación de la defensa.

Diagrama E.1. diagrama 2 servicios puertos dasc

